SIBYL: Speculative Inference Balancing for XPYD Scheduling in Disaggregated LLM Serving

Zihao Fan[†‡*], Changgang Zheng[§‡*], Bo Jiang[†□], Mingxin Li[§‡], Haonan Li[‡], Xiaoqing Sun[‡], Enge Song[‡], Shize Zhang[‡], Yang Song[‡], Xing Li[¶‡], Shunmin Zhu[‡□]
[†]Shanghai Jiao Tong University [‡]Alibaba Cloud [§]Nanjing University [¶]Zhejiang University

Abstract

Disaggregated serving is widely adopted for large language model inference, where requests are assigned to decode instances at prefill start to enable overlapping of KV-cache transmission with computation. The assignment typically aims to maximize performance based on the current loads of decode instances. However, when a request finishes prefilling, the actual load on its assigned decode instance usually differs from that at the time of decision-making. This load discrepancy *makes the assignment suboptimal and often leads to elevated tail latency that violates service-level objectives. The discrepancy stems from two sources: (1) backlog requests may finish decoding before the new request arrives at the assigned instance; and (2) requests currently undergoing prefilling may transition to decoding at any moment. To address this issue, we propose SIBYL, a speculative scheduler that projects per-instance load at the handoff moment and assigns each request to the instance with the least projected load. The projection is driven by a probabilistic workload model that jointly captures the residual lifetime of active tasks and the shadow interference introduced by in-flight prefills. We implement SIBYL as a lightweight, engine-agnostic proxy that requires no modification to existing inference engines. Experiments on real clusters and large-scale simulations show that SIBYL reduces decoding tail latency by up to* 56.9% *and lifts the optimal-assignment ratio from* 43.2% *to* 89.8% *over state-of-the-art approaches.*

1 Introduction

The rapid adoption of large language models (LLMs) [4, 12, 28, 29, 34, 48, 49, 59, 64] has driven the need for high-throughput, low-latency inference systems. LLM inference proceeds in two stages: (1) *prefill*, which processes the full prompt to initialize the key-value cache (KV cache), and (2) *decode*, which autoregressively generates tokens while reading the KV cache. To improve resource utilization and scalability, many modern serving frameworks [7, 20, 31, 35, 36, 39, 65] adopt *disaggregated serving*, which separates prefill and decode into two instance[1] pools with X prefill instances and Y decode instances (XPYD): prefill instances process prompts and produce KV caches, while decode instances consume these caches to generate tokens. The separation of prefill and decode eliminates the mutual interference in mixed workloads, where bursty prefills and concurrent decodes mutually stall each other [1]. This disaggregated design is critical for production-scale serving to meet service level objectives (SLOs) for both time-to-first-token (TTFT) and time-per-output-token (TPOT).

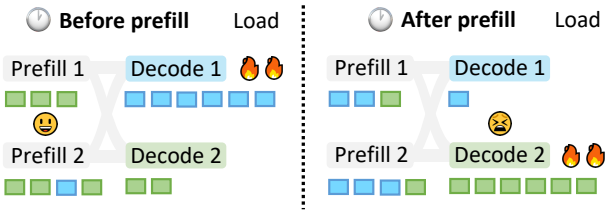


Figure 1. The load discrepancy in disaggregated serving. *Each colored block denotes a request, and the color indicates the decode instance it is bound to before prefill. An assignment at prefill start (Left) becomes suboptimal at decode start (Right) as the cluster state evolves during prefill, causing load imbalance.*

Despite the architectural benefits, disaggregated clusters frequently suffer from severe tail latency, causing SLO violations [53, 62]. This tail-latency issue stems from load imbalance: some instances become overloaded while others remain underutilized. One important source of this imbalance is a fundamental *load discrepancy* introduced by early assignment, which aims to overlap KV cache transmission with computation [35, 36]. This means the scheduler must choose a target decode instance at prefill start, while decoding only begins at the handoff moment (when prefill completes), risking placement on an instance that becomes overloaded during the prefill interval. As illustrated in Figure 1, a scheduler relying on the instantaneous snapshot at prefill start (Left) routes the request to the seemingly idle Decode 2. By the time decoding begins (Right), however, the load distribution has changed. Concurrent arrivals have increased the load on Decode 2, while requests on Decode 1 have completed. As a result, the original assignment is no longer load-balanced and becomes suboptimal.

Existing schedulers fail to mitigate this discrepancy because they rely on instantaneous metrics that become outdated during the prefill interval [13, 20, 24, 31, 39]. Addressing this problem requires shifting from reactive placement

[1]Throughout this paper, an instance hosts a full model replica.

* Equal contribution. □ Co-corresponding authors.

to predictive look-ahead. However, modeling the exact cluster state at the future handoff moment is non-trivial due to two distinct sources of uncertainty: (1) *Residual Uncertainty*, arising from the remaining lifetime of currently active tasks, which is highly stochastic, especially with the variance in generation lengths; and (2) *Shadow Interference*, arising from requests currently undergoing prefill, which constitute a latent workload that will contend for decode resources at the future handoff moment. Navigating these uncertainties to enable precise early assignment remains an unresolved challenge.

To mitigate the load discrepancy, we introduce Sibyl, a speculative scheduler for disaggregated serving. Instead of relying on instantaneous snapshots, Sibyl maintains a fine-grained *probabilistic workload model* that tracks every active and pending request and forecasts how each decode instance's occupancy evolves over time. The model decomposes the projected load at the future handoff moment τ into three categories: (i) *active tasks* currently decoding, whose residual lifetime at τ is captured via survival analysis; (ii) *pre-handoff shadow tasks* that are still in prefill but are projected to enter decoding before the new request; and (iii) *post-handoff shadow tasks*, prefilling requests whose decoding will overlap with the new request shortly after handoff. For each task, the projected load is expressed as the product of a *physical pressure* term (its KV-cache footprint at τ) and a *survival probability* (the conditional probability of remaining active at τ), estimated online via a discretized survival function that adapts to workload drift without retraining. On each request arrival, Sibyl estimates the prefill duration, queries this model to project per-instance loads at the predicted handoff, and binds the request to the instance with the minimum projected load, thereby preserving the KV-cache transfer-overlap benefits of early assignment while avoiding the load discrepancy pitfalls. We realize Sibyl as a lightweight, framework-agnostic proxy that integrates with mainstream inference engines such as vLLM and SGLang [20, 39] without modifying the internal executors, making speculative scheduling readily deployable in black-box production environments. Across real-cluster experiments and large-scale simulations, Sibyl cuts TPOT tail latency by up to 56.9% over state-of-the-art baselines and lifts the optimal-assignment ratio from 43.2% to 89.8% on heavy-tailed workloads.

We make the following contributions:

- We identify a *load discrepancy* between scheduling (prefill start) and execution (decode start) as a key contributor to decode-side tail latency in disaggregated serving. We further attribute this discrepancy to two sources: residual uncertainty of active tasks and shadow interference from in-flight prefills.
- We develop a probabilistic workload model that decomposes projected decode load into active, pre-handoff shadow, and post-handoff shadow components. Each component is modeled as a physical-pressure term weighted by an online-estimated survival probability, enabling robust scheduling under the heavy-tailed uncertainty of LLM generation lengths.
- We implement Sibyl as a lightweight, framework-agnostic scheduler proxy that integrates with mainstream inference engines without invasive modifications, making speculative early assignment practical for black-box production deployments.
- We evaluate Sibyl through real-cluster experiments and large-scale simulations with up to 128 decode instances. Sibyl reduces P99/P99.9 TPOT tail latency by up to 56.9% over state-of-the-art baselines and increases the optimal-assignment ratio from 43.2% to 89.8% on heavy-tailed workloads.

2 Background and Motivation

2.1 The Heterogeneity of Disaggregated Serving

LLM inference proceeds in two phases. Given an input prompt of n tokens $x_{1:n} = (x_1, \ldots, x_n)$, the *prefill* phase processes all tokens in parallel and produces the per-layer key/value tensors collectively known as the KV cache:

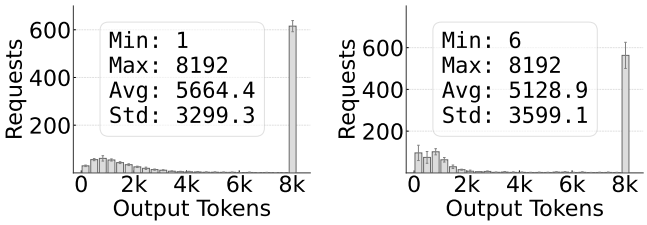
$$\mathrm{KV}_{1:n} = f_{\mathrm{prefill}}(x_{1:n}). \tag{1}$$

Per-layer computation is dominated by dense matrix multiplications over all n tokens at once, so prefill is *compute-bound* and bursty. The *decode* phase then autoregressively emits output tokens one at a time. At step t, given the existing KV cache, the model produces the next token

$$x_{n+t} = f_{\mathrm{decode}}\big(x_{n+t-1}, \mathrm{KV}_{1:n+t-1}\big), \tag{2}$$

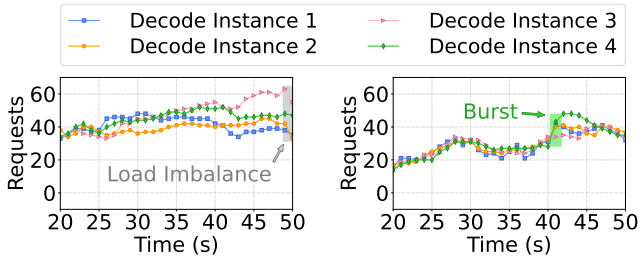
and the newly produced K, V entries are appended to the cache. Per-step computation is small and roughly constant, whereas memory traffic for reading the KV cache grows linearly with the running context length $n + t$, making decode *memory-bandwidth bound* and latency-sensitive [16, 17]. These contrasting hardware profiles are reflected in the two SLO metrics that govern serving quality: TTFT, determined by prefill latency, and TPOT, determined by per-step decode latency.

To exploit this heterogeneity, modern serving systems [18, 27, 31, 35, 36, 44, 65] adopt *disaggregated serving*, separating the two phases into dedicated instance pools backed by heterogeneous hardware tailored to each phase's characteristics. We consider an *XPYD* cluster comprising X prefill instances $\{P_1, \ldots, P_X\}$ and a set $\mathcal{D} = \{D_1, \ldots, D_Y\}$ of Y decode instances, each hosting a full model replica. Upon arrival, a request k is first scheduled to a prefill–decode pair (P_i, D_j), executes its prefill on P_i, has its KV cache streamed to D_j in parallel with the prefill computation, and finally generates output tokens on D_j [35, 36]. Crucially, the assignment (P_i, D_j) is committed at scheduling time, before prefill starts, so that KV cache transmission can be overlapped with prefill



(a) ShareGPT **(b)** Zeta

Figure 2. Output lengths on different datasets using Qwen3-32B [59]. *The workloads are evaluated with reasoning enabled and a maximum output limit of 8192 tokens.*



(a) RR Running Requests **(b)** ILS Running Requests

Figure 3. Limitations of baseline scheduling policies under a ShareGPT workload. *(a) RR fails to account for request heterogeneity, leading to significant load imbalance. (b) ILS ignores the prefill-decode time gap, blindly routing multiple requests to the same seemingly idle instance, which causes transient load spikes.*

computation rather than serialized after it. This architecture confines compute-heavy prefill bursts to the prefill pool and shields the latency-sensitive decode pool $\mathcal{D}$ from compute interference, while letting X and Y scale independently along their respective SLOs. These advantages have driven its widespread adoption in production environments [7, 36, 57].

2.2 The Load Discrepancy in Disaggregated Serving

Disaggregated serving introduces a *load discrepancy* between scheduling and execution. To make KV-cache transmission in parallel with computation, requests must be assigned to a decode instance before prefill starts. The load of decode instances, however, can change drastically after the prefill period, due to the influence of "uncertain" evolving loads. We identify two sources of this uncertainty:

(1) Residual Uncertainty from Active Tasks. At the moment of scheduling, instances host ongoing decode tasks with uncertain remaining lifetimes. This is further exacerbated by the emergence of reasoning-intensive models (*e.g.,* Chain-of-Thought [33, 54]), where generation length is dictated by the complexity of the logic chain rather than simple conversational patterns. We validate this using Qwen3-32B [59], as shown in Figure 2. The workloads exhibit extreme variance driven by reasoning steps: ShareGPT [41] displays a massive deviation ($\sigma \approx 3317$) with an average output length of 5657 tokens, frequently saturating the 8192 token limit. The Zeta dataset [60] exhibits a similarly large standard deviation ($\sigma \approx 3492$). As a result, residual decode load is inherently difficult to predict, often leading to severe errors when estimating future load.

(2) Interference from Shadow Tasks. Equally critical is the *shadow workload*, which refers to requests assigned to a decode instance but still currently undergoing prefill. Although not yet decoding, these tasks impose two distinct risks to the given new request upon decoding: tasks transitioning to the decode phase *prior* to the new request (impose immediate contention) and those joining the decode phase during mid-execution (introduce delayed interference). Failing to account for these risks blinds the scheduler to impending pressures, causing concurrent requests to converge on the same seemingly idle instance.

	RPS				
Metric	2	4	6	8	10
Load Shift Rate (%)	8.4	13.0	22.0	28.5	36.1
Bind Error Rate (%)	21.3	29.2	34.5	40.0	42.4

Table 1. ILS suffers from substantial load shift and binding errors as the request rate grows on a ShareGPT workload. *(1) Load shift rate: the average relative change in load on the ILS-bound instance between prefill start and handoff. (2) Bind error rate: the fraction of requests whose bound instance is no longer the least-loaded one at handoff.*

2.3 Limitations of Existing Solutions

Most existing schedulers are myopic, relying on static rules or instantaneous load snapshots, and thus become unreliable under stochastic, fast-evolving workloads (Figure 3). Migration can partially correct imbalance, but it is reactive and introduces non-trivial transfer overhead. We next summarize representative strategies and their limitations.

(1) Round-Robin (RR). Being load-agnostic, RR [20] fails to accommodate the heavy-tailed heterogeneity of LLM generation. As shown in Figure 3(a), the stochastic arrival of long-decode tasks can lead to disproportionate backlog accumulation on specific instances. Since RR continues to blindly route requests to these instances, it exacerbates load imbalance and leads to severe tail latency.

(2) Instantaneous Load Scheduling (ILS). Strategies such as least outstanding requests [24, 31] route new tasks to the instance with the minimum instantaneous load. Because the snapshot used at prefill start becomes stale during the prefill interval, ILS suffers from two parallel failure modes that are both manifestations of this myopia. *First, each individual binding decision is unreliable*: the bound instance is

Model	Attn.	CW	KV/tok	KV@CW	Mig.
Qwen3-32B [59]	GQA†	128K	256 KB	32.0 GB	172 ms
Qwen3-235B-A22B [59]	GQA	128K	188 KB	23.5 GB	126 ms
Hy3-preview (300B) [51]	GQA	256K	320 KB	80.0 GB	400 ms
DeepSeek-V3 (671B) [7]	MLA‡	128K	34.3 KB	4.30 GB	23 ms
DeepSeek-V4 (1.6T) [6]	CSA§+HCA¶	1M	4.8 KB	4.81 GB	24 ms

†Grouped-Query Attention; ‡Multi-head Latent Attention; §Compressed Sparse Attention; ¶Heavily Compressed Attention.

Table 2. KV-cache volume and migration time for a single in-flight request at the model's maximum context window (CW). *KV-cache stored at native precision (BF16 for Qwen3 and Hy3, FP8 for DeepSeek).*

frequently no longer the least-loaded one by handoff. We quantify this on a ShareGPT trace using two metrics (Table 1): the *load shift rate* measures the average relative change in load on the ILS-bound instance between prefill start and handoff, while the *bind error rate* measures the fraction of requests whose bound instance is no longer the least-loaded one at handoff. Both rise sharply with the request rate: at 10 RPS, the load on the bound instance shifts by 36% on average, and ILS picks the wrong instance for 42% of requests. *Second, concurrent decisions become correlated under bursty traffic*: because all in-flight requests are routed against the same stale snapshot, they converge onto the same seemingly idle instance, producing the sharp congestion spikes visible in Figure 3(b).

(3) Rescheduling and Migration. While migration mechanisms [47, 53] can mitigate imbalance, they are inherently reactive and costly. Migrating an in-flight request requires transferring its *entire* KV cache across nodes, whose volume grows linearly with the running context length. Table 2 quantifies this for representative models at their maximum context window: a single long-context request can require migrating tens of GBs of KV state, reaching 32 GB for Qwen3-32B and 4.8 GB for DeepSeek-V4, while even aggressively compressed MLA models still require transferring several GBs. Even over a fast 200 GB/s inter-node link (4× ConnectX-7, a common decode-side configuration on H20-based servers), migration latency can exceed 100 ms for large-context requests, reaching up to 400 ms in our examples. Worse, these times are highly optimistic upper bounds: in practice the migration traffic contends fiercely with the incoming KV-cache traffic from prefill instances and with the expert-parallel (EP) all-to-all traffic inside the decode instance, so the effective stall is substantially larger. Yet even this best case already dwarfs the per-token TPOT SLO (typically ~50 ms), stalling the very request it aims to rescue. Beyond this bandwidth penalty, such mechanisms also mandate invasive modifications to engine internals, rendering them impractical for standard black-box deployments.

Overall, existing designs are either myopic or migration-heavy, motivating Sibyl as a proactive, lightweight method to mitigate load discrepancy in disaggregated serving.

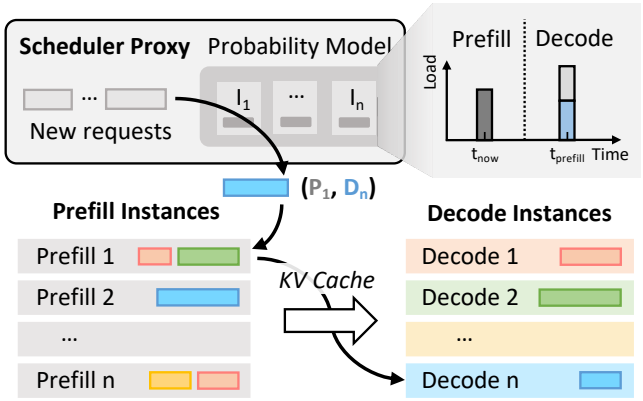


Figure 4. System overview of Sibyl. *The probability-driven global scheduler proxy can predict future decode loads. It assigns new requests to prefill (P_i) and decode (D_j) instances based on projected availability at the estimated handoff time.*

3 Design Overview

3.1 System Architecture

As shown in Figure 4, Sibyl interposes a thin *global scheduler proxy* on the control path of an existing XPYD cluster. For *transparency*, it separates control from data: prefill and decode instances run *unmodified* engines (*e.g.*, vLLM, SGLang [20, 39]) and stream KV caches peer-to-peer as before, while the proxy only observes lightweight metadata (prompt lengths and decode start/finish events) and emits one placement per request. Sibyl thus drops in as a black-box replacement for the cluster's stock load balancer, with no executor changes.

On each arrival, the proxy chains three steps. *(i)* It estimates the request's prefill duration, which is near-determ inistic given the prompt length [26, 57, 65], to pin down the future *handoff moment* τ at which the request will actually begin decoding. *(ii)* It consults a *probabilistic workload model* that, for every decode instance, predicts how busy that instance will be *at* τ, accounting both for requests still running then and for requests currently in prefill that will have joined decoding by then. *(iii)* It binds the request to the prefill–decode pair (P_i, D_j) whose decode instance is predicted to be *least loaded* at τ, and marks it as pending there so that simultaneous decisions stay consistent. Prefill then runs on P_i with its KV cache streamed to D_j in the background, and decoding starts at handoff. By deciding at prefill start yet optimizing for the predicted state at τ, Sibyl keeps the transfer-overlap benefit of early assignment while still balancing load at the moment decoding actually begins. This design, however, hinges on a single hard question: how to faithfully project the decode load at τ across the gap between when the decision is made (prefill start) and when it takes effect (handoff), an interval over which the cluster state can shift substantially. We next present the principle

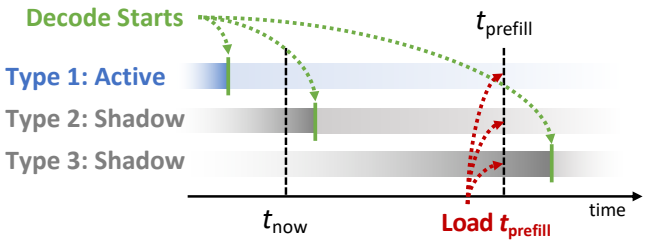


Figure 5. Speculative load projection. *The scheduler estimates instance occupancy at τ (the future handoff moment, also referred to as $t_{prefill}$), when the new task arrive at t_{now}, by aggregating the residual active load and the shadow load from pending tasks.*

by which Sibyl bridges this gap (§3.2), and formalize the resulting workload model (§4).

3.2 Design Philosophy: "Bridging" the Temporal Gap

To mitigate the load discrepancy, Sibyl optimizes for the projected cluster state at the future handoff moment τ rather than the current snapshot t_{now}. As shown in Figure 5, the scheduler aggregates potential load (introduced by three types of tasks) to identify the instance with the minimal projected load at the handoff moment τ:

- **Type 1: Active Tasks.** Requests currently being decoded. Their residual load at τ is stochastic, derived from the survival probability during prefill.
- **Type 2: Pre-Handoff Shadow.** Concurrent prefill tasks projected to start decoding *before* τ. These form hidden contention that is invisible at t_{now} but will be occupying resources at the handoff.
- **Type 3: Post-Handoff Shadow.** Concurrent prefill tasks projected to start decoding *after* τ. Although arriving later, they pose a potential interference risk that may constrain near-term availability.

By aggregating the projected impact of these workloads, Sibyl constructs a comprehensive view of the *future* load.

4 Probabilistic Workload Modeling

We establish a probabilistic framework to formulate the scheduling decision as a load minimization problem at the handoff moment. For reference, the key notations are summarized in Table 3.

4.1 Objective Function

Let $\mathcal{D}$ denote the set of decode instances. For a candidate instance $j \in \mathcal{D}$, the total projected load $\mathcal{L}_j(\tau)$ at the future handoff time τ is defined as the sum of the residual load from currently running tasks and the projected load from pending tasks (*i.e.,* requests currently undergoing prefill that are already bound to instance j):

$$\mathcal{L}_j(\tau) = \sum_{k \in Q_j^{\text{run}}} \mathcal{L}_{\text{I}}(k,\tau) + \sum_{k \in Q_j^{\text{pend}}} \left(\mathbb{I}_{\tau_k \le \tau} \mathcal{L}_{\text{II}}(k,\tau) + \mathbb{I}_{\tau_k > \tau} \mathcal{L}_{\text{III}}(k,\tau) \right) \tag{3}$$

where Q_j^{run} and Q_j^{pend} denote the sets of active and pending tasks on instance j, respectively. The terms $\mathcal{L}_{\text{I}}$, $\mathcal{L}_{\text{II}}$, and $\mathcal{L}_{\text{III}}$ correspond to the projected loads of active, pre-handoff shadow, and post-handoff shadow tasks, as detailed in §3.2. $\mathbb{I}_{\text{condition}}$ denotes the indicator function, which equals 1 if the condition holds and 0 otherwise. Based on this metric, the scheduler identifies the optimal instance

$$j^* = \arg\min_{j \in \mathcal{D}} \mathcal{L}_j(\tau), \tag{4}$$

thereby directing the request to the instance with the minimum projected load at the handoff moment. The following sections define the individual load terms $\mathcal{L}_{\text{I}}$, $\mathcal{L}_{\text{II}}$, and $\mathcal{L}_{\text{III}}$.

Symbol	Description
System State & Scheduling	
t_{now}	Current scheduling timestamp
τ	Projected handoff moment, $t_{\text{now}} + \delta_{\text{prefill}}$
$\mathcal{D}$, j	Decode instance set; candidate $j \in \mathcal{D}$
$Q_j^{\text{run}}, Q_j^{\text{pend}}$	Active / pending tasks on j
v_k, $\bar{v}_{\text{sys}}$	Per-task / system-average decode rate
Probabilistic Primitives	
L_k, l_k^{in}	Generation-length r.v.; input prompt length
$l_k(t)$	Generated length of task k at time t
$\mathcal{M}_k(\tau)$	Physical pressure: $l_k^{\text{in}} + l_k(\tau)$
$\mathcal{S}(x), \hat{\mathcal{S}}(x)$	Survival function and its online estimator
$\mathcal{P}_k(\tau)$	Conditional survival prob. of k at τ
Speculative Load Composition	
$\mathcal{L}_j(\tau)$	Total projected load on j (to minimize)
$\mathcal{L}_{\text{I,II,III}}(k,\tau)$	Loads from Type 1 / 2 / 3 tasks

Table 3. Key notations used in the probabilistic workload modeling.

4.2 Modeling Primitives

To compute $\mathcal{L}_{\text{I, II}}$ in Eq. (3), we introduce a mechanistic model built upon two primitives: *Physical Pressure* ($\mathcal{M}$) and *Survival Probability* ($\mathcal{P}$).

Primitive 1: Physical Pressure ($\mathcal{M}$). We define physical pressure $\mathcal{M}_k(\tau)$ as the request's total token occupancy to

quantify its hardware cost:

$$\mathcal{M}_k(\tau) = l_k^{\text{in}} + l_k(\tau) \tag{5}$$

where l_k^{in} denotes the prompt length and $l_k(\tau)$ represents the generated length at time τ. Since LLM decoding is memory-bandwidth bound [35, 65], the execution overhead is dominated by loading the KV cache, which scales linearly with the total sequence length. Thus, $\mathcal{M}$ effectively captures the hardware cost.

Primitive 2: Conditional Survival Probability ($\mathcal{P}$). This component quantifies the *likelihood* of a request remaining active at a specific future moment. Predicting exact generation lengths is inherently unstable [53, 55]. Instead, we model task completion uncertainty using the survival function, which directly captures the probability of a request remaining active beyond a given time. Let L_k denote the generation length of request k. We define the survival function as $\mathcal{S}(x) = \mathbb{P}(L_k > x)$ for a length threshold x, and assume that $\{L_k\}$ are i.i.d. samples drawn from a common distribution. For a request k with current generated length $l_k(t_{\text{now}})$, the probability $\mathcal{P}_k(\tau)$ that it survives up to the projected generated length $l_k(\tau)$ is:

$$\begin{aligned}\mathcal{P}_k(\tau) &= \mathbb{P}(L_k > l_k(\tau) \mid L_k > l_k(t_{\text{now}})) \\ &= \frac{\mathbb{P}(L_k > l_k(\tau))}{\mathbb{P}(L_k > l_k(t_{\text{now}}))} = \frac{\mathcal{S}(l_k(\tau))}{\mathcal{S}(l_k(t_{\text{now}}))}\end{aligned} \tag{6}$$

To compute Eq. (6) practically, we approximate the theoretical function $\mathcal{S}$ using an online estimator $\hat{\mathcal{S}}$. Instead of tracking every integer length, we discretize the output length space into coarse-grained intervals (buckets) with a fixed step size Δ. The estimator entries $\hat{\mathcal{S}}(l)$ are maintained only for discrete boundaries $l \in \{\Delta, 2\Delta, \dots\}$. Upon each request completion with final length L_{final}, we update the estimated probabilities for all relevant buckets ($l \le L_{\text{final}}$) using an exponential moving average (EMA):

$$\hat{\mathcal{S}}(l) \leftarrow \alpha \cdot \hat{\mathcal{S}}(l) + (1-\alpha) \cdot \mathbb{I}_{L_{\text{final}} \ge l} \tag{7}$$

where $\alpha \in (0,1)$ is the smoothing factor. This online update prioritizes recent observations, allowing the estimator to refine $\hat{\mathcal{S}}$ to reflect the current output length distribution.

4.3 Speculative Load Composition

Using these primitives, we define the projected load for tasks potentially active at τ (*i.e.,* Type 1 and Type 2) as its expected load at τ, as established in Eq. (8).

$$\mathcal{L}(k,\tau) = \mathcal{M}_k(\tau) \cdot \mathcal{P}_k(\tau), \qquad k \in \text{Type 1/2}. \tag{8}$$

Derivation. We define the system load at time τ as the aggregate decoding load contributed by all active tasks:

$$\mathbb{L}(\tau) = \sum_k \mathcal{M}_k(\tau)\, \mathbb{I}_{(\tau_k \le \tau,\, T_k > \tau)}, \tag{9}$$

where τ_k and T_k denote the decode start time and completion time of task k, respectively. The indicator function ensures that only tasks that have started decoding but have not yet completed contribute to the instantaneous load.

Conditioned on the system state at the current time t_0, taking expectation on both sides yields

$$\mathbb{E}_0[\mathbb{L}(\tau)] = \sum_k \mathbb{E}_0\left[\mathcal{M}_k(\tau)\, \mathbb{I}_{(\tau_k \le \tau,\, T_k > \tau)}\right]. \tag{10}$$

For an active decoding task, the physical pressure evolves linearly over the short prediction interval:

$$\mathcal{M}_k(\tau) = l_{k,0} + v_k(\tau - t_0), \tag{11}$$

where $l_{k,0}$ is the accumulated load at time t_0 and v_k is the observed decoding rate. Since both quantities are deterministic conditioned on the current system state, we can factor them out of the expectation:

$$\mathbb{E}_0[\mathbb{L}(\tau)] = \sum_k \big(l_{k,0} + v_k(\tau - t_0)\big)\, \mathbb{P}_0(T_k > \tau \mid \tau_k \le \tau). \tag{12}$$

Next, observe that a task survives beyond time τ if and only if its total decoding length exceeds the generated length at τ. Therefore,

$$\mathbb{P}_0(T_k > \tau) = \mathbb{P}_0(L_k > l_k(\tau)), \tag{13}$$

where L_k denotes the random total decoding length of task k. Substituting this relation into Eq. (12), we obtain

$$\mathbb{E}_0[\mathbb{L}(\tau)] = \sum_k \underbrace{\big(l_{k,0} + v_k(\tau - t_0)\big)}_{\mathcal{M}_k(\tau)} \cdot \underbrace{\mathbb{P}_0(L_k > l_k(\tau))}_{\mathcal{P}_k(\tau)}, \tag{14}$$

which yields Eq. (8).

For Type 1 and Type 2 tasks, the formulation differs only in the reference state used for projection (*i.e.,* the choice of t_0, $l_{k,0}$, and v_k); the expected-load expression in (14) remains identical. In contrast, Type 3 tasks do not begin decoding until after τ, contributing zero instantaneous load at the handoff. Consequently, instead of applying Eq. (8), we model $\mathcal{L}_{\text{III}}(k,\tau)$ as a lookahead interference penalty to capture near-future contention.

4.3.1 Type 1: Active Tasks ($t_{\text{start}} \le t_{\text{now}}$).

For tasks already in decoding, we leverage their observed execution statistics for projection. We project the token count using the task's specific decoding rate v_k, and compute the conditional survival probability directly from our estimator $\hat{\mathcal{S}}$:

$$\mathcal{L}_{\text{I}}(k,\tau) = \underbrace{(l_k^{\text{in}} + l_k(\tau))}_{\mathcal{M}} \cdot \underbrace{\frac{\hat{\mathcal{S}}(l_k(\tau))}{\hat{\mathcal{S}}(l_k(t_{\text{now}}))}}_{\mathcal{P}} \tag{15}$$

where $l_k(\tau) = l_k(t_{\text{now}}) + v_k \cdot (\tau - t_{\text{now}})$ is obtained by linearly projecting the generated length using the task's observed effective decoding rate v_k[2] over the short interval $[t_{\text{now}}, \tau]$. The probability term $\mathcal{P}$ captures the likelihood that task k remains active from its current generation length $l_k(t_{\text{now}})$ to the projected generation length $l_k(\tau)$.

[2]Calculated as generated length divided by decoding time.

Algorithm 1: SIBYL

Input: Request r, Time t_{now}, Instances $\mathcal{D}$, Estimator $\hat{\mathcal{S}}$

1 **Function** `ScheduleRequest(`r, t`)`**:**
2 $\quad\tau \leftarrow t_{\text{now}} + \text{EstPrefill}(r.length)$;
3 $\quad$**for** *each instance* $j \in \mathcal{D}$ **do**
4 $\quad\quad\mathcal{L}_j, \mathcal{L}_{\text{I}}, \mathcal{L}_{\text{II}}, \mathcal{L}_{\text{III}} \leftarrow 0$;
5 $\quad\quad$**for** $k \in Q_j^{run}$ **do**
6 $\quad\quad\quad l_k(\tau) \leftarrow l_k(t_{\text{now}}) + v_k \cdot (\tau - t_{\text{now}})$;
7 $\quad\quad\quad\mathcal{M} \leftarrow l_k^{\text{in}} + l_k(\tau)$;
8 $\quad\quad\quad\mathcal{P} \leftarrow \hat{\mathcal{S}}(l_k(\tau))/\hat{\mathcal{S}}(l_k(t_{\text{now}}))$;
9 $\quad\quad\quad\mathcal{L}_{\text{I}} \leftarrow \mathcal{L}_{\text{I}} + \mathcal{M} \times \mathcal{P}$;
10 $\quad\quad$**for** $k \in Q_j^{pend}$ **do**
11 $\quad\quad\quad\tau_k \leftarrow k.arrival + \text{EstPrefill}(k.length)$;
12 $\quad\quad\quad\Delta_{\text{gap}} \leftarrow (\tau - \tau_k) \cdot \bar{v}_{sys}$;
13 $\quad\quad\quad$**if** $\Delta_{gap} > 0$ **then**
14 $\quad\quad\quad\quad\mathcal{M} \leftarrow l_k^{\text{in}} + \Delta_{\text{gap}}$;
15 $\quad\quad\quad\quad\mathcal{P} \leftarrow \hat{\mathcal{S}}(\Delta_{\text{gap}})$;
16 $\quad\quad\quad\quad\mathcal{L}_{\text{II}} \leftarrow \mathcal{L}_{\text{II}} + \mathcal{M} \times \mathcal{P}$
17 $\quad\quad\quad$**else**
18 $\quad\quad\quad\quad\mathcal{L}_{\text{III}} \leftarrow \mathcal{L}_{\text{III}} + \text{ReLU}(l_k^{\text{in}} + \Delta_{\text{gap}})$;
19 $\quad\quad\mathcal{L}_j \leftarrow \mathcal{L}_{\text{I}} + \mathcal{L}_{\text{II}} + \mathcal{L}_{\text{III}}$;
20 $\quad$**return** $\arg\min_j \mathcal{L}_j$;

4.3.2 Type 2: Pre-Handoff Shadow Tasks ($\tau_k \le \tau$).

Let τ_k denote the projected start time of task k. We model pending tasks satisfying $\tau_k \le \tau$ as pre-handoff shadow tasks. Since these tasks become active before the new request arrives, they will occupy resources at moment τ. Unlike Type 1 tasks where per-task decoding rate is observable, pending tasks lack runtime history. We rely on the system-wide average speed $\bar{v}_{\text{sys}}$ to estimate the potential resource consumption, and define the projected generated length as $l_k(\tau) = (\tau - \tau_k) \cdot \bar{v}_{\text{sys}}$. For Type 2 tasks, the probability term is computed from the start state at τ_k, where $l_k(\tau_k) = 0$. Thus, the conditional survival probability up to $l_k(\tau)$ reduces to $\hat{\mathcal{S}}(l_k(\tau))/\hat{\mathcal{S}}(0)$, which simplifies to $\hat{\mathcal{S}}(l_k(\tau))$ since $\hat{\mathcal{S}}(0) = 1$. Thus, the expected load for Type 2 tasks is:

$$\mathcal{L}_{\text{II}}(k, \tau) = \underbrace{(l_k^{\text{in}} + l_k(\tau))}_{\mathcal{M}} \cdot \underbrace{\hat{\mathcal{S}}(l_k(\tau))}_{\mathcal{P}} \tag{16}$$

4.3.3 Type 3: Post-Handoff Shadow Tasks ($\tau_k > \tau$).

For tasks projected to start after the handoff, we use an *interference-decay* model to capture their near-future contention with the post-handoff execution window. Although these tasks are inactive at the handoff moment, they will start decoding shortly after and may overlap with the decode of the current request, thereby introducing additional interference. We model this interference by treating the prompt length l_k^{in} as the base interference. We then linearly discount this based on the temporal distance $(\tau_k - \tau)$, using the system-average decode rate $\bar{v}_{\text{sys}}$. Intuitively, tasks starting soon after τ have limited temporal slack and are more likely to overlap with ongoing requests, whereas later arrivals experience weaker contention. We therefore clamp the discounted value at zero to ignore sufficiently distant arrivals:

$$\mathcal{L}_{\text{III}}(k, \tau) = \text{ReLU}\big(l_k^{\text{in}} - \bar{v}_{\text{sys}}\,(\tau_k - \tau)\big). \tag{17}$$

4.4 Scheduling Algorithm

Algorithm 1 outlines the scheduling procedure and runs in $O(|\mathcal{D}| + N_{\text{total}})$ time. Here, $|\mathcal{D}|$ is the number of candidate decode instances, and N_{total} is the number of requests traversed for load aggregation (active decodes and pending shadows). The procedure begins by establishing the target handoff time τ using the new request's estimated prefill latency (Line 2). For each candidate instance, the algorithm then aggregates the projected load from two distinct sources. First, it projects the state of currently active tasks (Lines 6–9), weighting their estimated physical pressure by the corresponding conditional survival probability; Second, it evaluates pending shadow tasks (Lines 11–18) by estimating their start times and separately handling: Type 2 expected loads ($\tau_k \le \tau$) and Type 3 future-load penalties ($\tau_k > \tau$). Finally, the algorithm selects the instance that minimizes the resulting cumulative congestion (Line 20).

5 Implementation

SIBYL is implemented as a lightweight, non-intrusive middleware that operates entirely in the control plane. This framework-agnostic design allows SIBYL to be deployed alongside mainstream inference engines, such as vLLM and SGLang [20, 39], without requiring invasive modifications to the underlying kernels or model executors.

Standalone Architecture. Similar to the reference implementation in vLLM's disaggregated serving example,[3] SIBYL functions as a standalone HTTP proxy. This decoupled architecture allows SIBYL to manage request routing and scheduling decisions centrally while maintaining compatibility with standard OpenAI-compatible API interfaces.

State Synchronization. To perform speculative scheduling, SIBYL maintains a real-time view of the cluster state. Instead of relying on complex protocols, SIBYL utilizes the native monitoring interfaces provided by vLLM. Specifically, the proxy periodically polls the `/metrics` endpoint of each decode instance via HTTP `GET` requests. By parsing these metrics, which include the number of running requests, KV cache usage, and iteration stats, SIBYL updates its internal probabilistic workload model with negligible overhead.

[3] Specifically, `disagg_proxy_p2p_nccl_xpyd.py`.

Prefill Time Estimation. Projecting the handoff moment τ hinges on estimating each request's prefill completion time, a primitive shown to be practical at production scale (*e.g.,* Aegaeon [57] on Alibaba Cloud). Sibyl exploits the regular structure of production XPYD deployments, where prefill instances run with `batch_size=1` and chunked prefill, so every step processes a fixed `chunk_size` tokens; our experiments adopt the same configuration. Under non-prefill-dominated workloads, the per-step latency varies little, making prefill timing reliable to predict. A request of L prompt tokens thus takes $\lceil L/\texttt{chunk\_size} \rceil$ steps, completing after its own steps plus those queued ahead on the same instance. We profile the per-step latency offline and fit a low-degree polynomial ($R^2 = 0.998$; Appendix C.4).

Enabling Reasoning in Benchmarks. Standard benchmarking tools (*e.g.,* `vllm bench`) often lack native support for passing engine-specific parameters required by reasoning models. To evaluate reasoning capabilities (*e.g.,* Chain-of-Thought) systematically, we implemented a request injection mechanism within the Sibyl proxy. For every incoming inference request, the proxy intercepts the JSON payload and automatically injects the necessary configuration to trigger the model's reasoning mode. Specifically, we modify the request body to include:

```
request_data["extra_body"]["chat_template_kwargs"]=
    {"enable_thinking": True}
```

This ensures that all requests processed during our evaluation, regardless of the client-side configuration, correctly trigger the reasoning logic in the reasoning model.

6 Evaluation

6.1 Experimental Setup

We first summarize the experimental settings in this section. Full details are provided in Appendix C.

Testbed. We conduct our evaluation on an NVIDIA H800-based cluster. For all datasets, we deploy a 4P4D disaggregated configuration, with four prefill instances and four decode instances. In all scenarios, each GPU hosts a replica of a single model. For the interconnect, we leverage vLLM's native `P2pNcclConnector` [32] to enable efficient layer-wise KV cache transmission between prefill and decode instances.

Models. We evaluate Sibyl using Qwen3-8B [59], a widely adopted open-weights model in production environments. This model represents a typical single-GPU workload that fits comfortably within the H800's on-device memory (80 GB), eliminating the communication overhead of tensor parallelism [43]. To assess generality across model scales and families, we further evaluate Sibyl on three additional open-weights models: Qwen3-32B [59], Qwen2.5-7B-Instruct [50], and Llama-3-8B-Instruct [2], with full results deferred to Appendix D.

Datasets. We evaluate Sibyl across four distinct datasets to cover a range of inference scenarios:

- `Random`, a synthetic workload generated by the built-in `vLLM bench` benchmark with uniformly distributed input and output lengths, used for controlled micro-benchmarking of system performance.
- ShareGPT [41], a de facto standard dataset comprising real-world user conversations, exhibiting the heavy-tailed length distribution typical of chat services.
- Zeta [60], a dataset of predictive code-editing tasks from Zed Industries, used to assess performance on specialized programming assistant workloads.
- NuminaMath [21], a large-scale collection of competition-style mathematics problems paired with chain-of-thought solutions, which stresses Sibyl under reasoning-intensive workloads with long and highly variable output lengths.

Together, these datasets cover a broad spectrum of mainstream LLM serving scenarios, including synthetic micro-benchmarking, conversational chat, programming assistance, and mathematical reasoning.

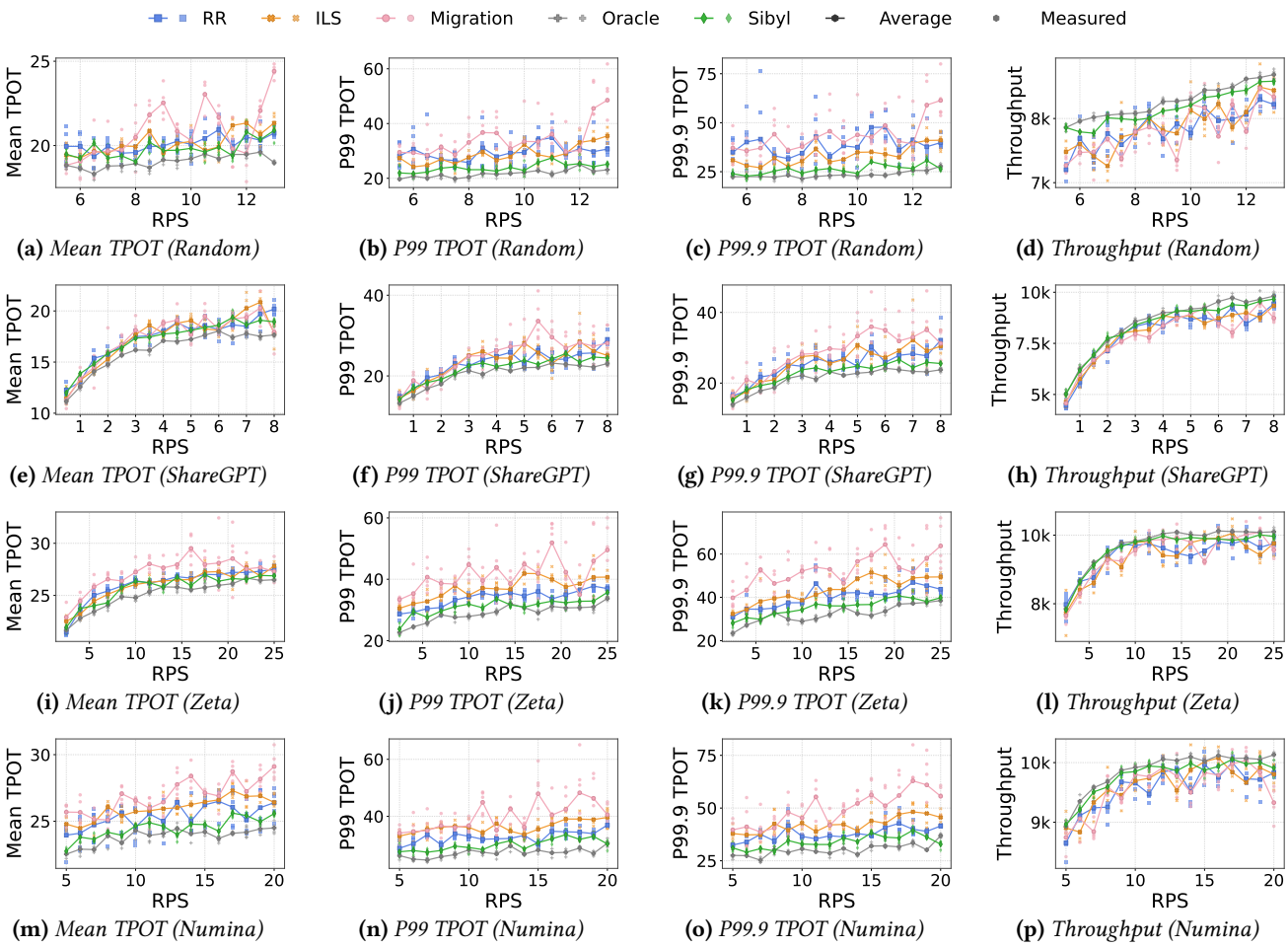
Baselines. We compare Sibyl against four representative policies that span the design spectrum from load-agnostic dispatch to oracle-guided scheduling:

- *Round-Robin (RR)* [20], a static, load-agnostic approach that dispatches requests cyclically, which is also the default scheduling policy adopted in serving frameworks such as vLLM and SGLang.
- *Instantaneous Load Scheduling (ILS)* [24, 31], a greedy policy that routes each request to the instance with the fewest active requests.
- *Migration* [47, 53], a reactive rebalancing baseline that adapts the runtime migration mechanism of Llumnix [47] to the disaggregated setting, transferring KV caches from overloaded to under-utilized decode instances after early assignment.
- *Oracle*, an idealized variant of Sibyl that replaces the survival-based residual-lifetime estimator with the ground-truth remaining decode length of every active task, serving as the practical performance upper bound of our framework.

6.2 Overall System Performance

Figure 6 reports the end-to-end performance of Sibyl against the three deployable baselines (RR, ILS, and Migration) on four workloads, together with the idealized Oracle variant as an upper bound. For every metric, we report the *average / maximum* percentage improvement of Sibyl (or Oracle) over the corresponding baseline across all tested request rates.

Tail latency. Sibyl's gains are most pronounced at the tail, where load-discrepancy-induced hotspots translate directly into SLO violations. On the bursty Random workload, where concurrent prefills repeatedly target the same seemingly idle decode instance, Sibyl reduces P99 / P99.9



(a) *Mean TPOT (Random)* **(b)** *P99 TPOT (Random)* **(c)** *P99.9 TPOT (Random)* **(d)** *Throughput (Random)*

(e) *Mean TPOT (ShareGPT)* **(f)** *P99 TPOT (ShareGPT)* **(g)** *P99.9 TPOT (ShareGPT)* **(h)** *Throughput (ShareGPT)*

(i) *Mean TPOT (Zeta)* **(j)** *P99 TPOT (Zeta)* **(k)** *P99.9 TPOT (Zeta)* **(l)** *Throughput (Zeta)*

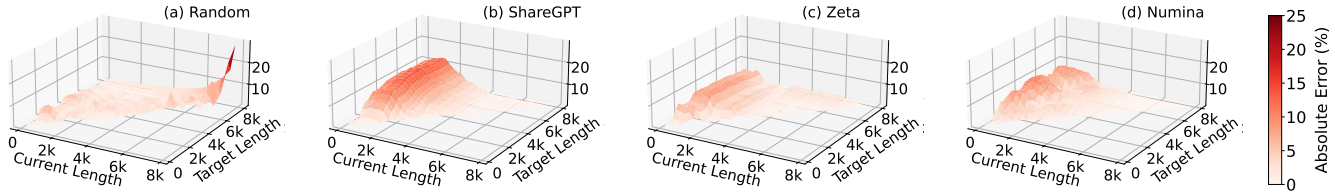
(m) *Mean TPOT (Numina)* **(n)** *P99 TPOT (Numina)* **(o)** *P99.9 TPOT (Numina)* **(p)** *Throughput (Numina)*

Figure 6. *End-to-end performance comparison among five methods across four datasets. TPOT is measured in milliseconds (ms), and throughput in tokens per second (tps).*

TPOT by 18.9%/31.5% on average against RR (with maxima of 29.2%/43.6%), by 16.2%/20.5% against ILS (max 29.3%/35.6%), and by 26.6%/38.0% against Migration, peaking at 48.2%/56.9%. The same pattern carries over to the heavy-tailed ShareGPT, the code-editing Zeta, and the reasoning-intensive Numina-Math traces: Sibyl's average P99.9 TPOT reductions over Migration reach 18.5%, 32.1%, and 30.6% respectively, with maxima ranging from 32.8% to 43.4%. Notably, Sibyl's lead over Migration is consistently larger than its lead over RR or ILS on every workload, indicating that reactive KV-cache transfer incurs substantial migration overhead and can even amplify tail latency under bursty contention, whereas proactive look-ahead avoids these costly corrections altogether.

Mean latency and throughput. The same direction holds for average latency and cluster capacity. Sibyl reduces Mean TPOT by 0.5% to 5.8% on average over RR and ILS, and by 1.0% to 9.0% over Migration, with the largest savings (9.0%/13.5% avg / max vs. Migration) appearing on Numina-Math, where chain-of-thought generation amplifies residual uncertainty and exposes the limits of reactive baselines. At the same time, throughput improves by up to 12.9% over RR, 10.4% over ILS, and 16.9% over Migration, showing that Sibyl suppresses tail latency without sacrificing cluster capacity.

How close is Sibyl to the upper bound? For this comparison, we define the reference baseline at each request rate as the mean of RR, ILS, and Migration on the metric of interest, and report the percentage improvement of Sibyl (or Oracle) over this baseline averaged and maximized across all tested rates. Under this aggregation, Oracle, which uses ground-truth decode lengths, improves Mean / P99 / P99.9 TPOT by 6.7%/22.3%/29.1% on average (with maxima of 11.8%/30.1%/37.4%), and throughput by 4.5% on average (7.4% at maximum). Sibyl, without any such oracle access, already attains 3.1%/15.6%/21.8% on the latency metrics and 3.3% on throughput. Translated into raw performance, Sibyl sits within 3.9% of Oracle on Mean TPOT, 8.9% on P99 TPOT, 10.7% on P99.9 TPOT, and only 1.1% on throughput, recovering the majority of Oracle's achievable improvement over the



(a) Random **(b)** ShareGPT **(c)** Zeta **(d)** Numina

Figure 7. *Calibration error of survival probability estimation.*

deployable baselines. This indicates that the survival-based estimator is sufficiently accurate for practical scheduling, and that the dominant source of remaining error is the fundamental variance of LLM generation lengths rather than the algorithm itself. We further evaluate Sibyl under different model and hardware configurations, with additional results provided in Appendix D.

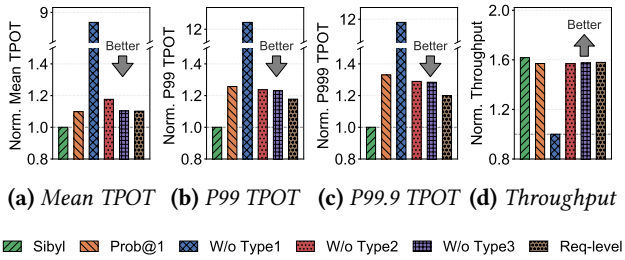
6.3 Estimation Error of Survival Probability

To validate the estimation error of Sibyl's profiling mechanism, we conduct the following analysis. We performe a static evaluation using the checkpoints trained on the Random, ShareGPT, Zeta and Numina datasets. Specifically, we queried the estimator on a dense grid of current lengths and target lengths across the entire trace, calculating the mean absolute error between the predicted survival probability $\mathcal{P}$ and ground-truth conditional probabilities derived from the cumulative distribution functions of the test datasets.

Figure 7 visualizes the error surface across the prediction space. Overall, Sibyl demonstrates robust calibration quality across diverse workload distributions. The Random workload shows negligible estimation error consistent with a uniform distribution, except for minor boundary effects at extreme context lengths. Even under the high output variance of ShareGPT, the estimator maintains an absolute error below 14%, with the vast majority of the configuration space exhibiting $< 5\%$ deviation. Similarly, for the coding-intensive Zeta dataset, the error remains under 9%. On the math-reasoning Numina dataset, whose CoT completions exhibit a heavy-tailed length distribution, the estimator achieves an absolute error below roughly 11%, with predictions becoming nearly exact once requests exceed ~2.5K context tokens.

6.4 Sensitivity Analysis

To validate Sibyl's design choices, we conduct a sensitivity analysis that selectively disables individual strategies or alters the load metric. Detailed methodology and variant definitions are provided in Appendix F. Figure 8 summarizes the results; for each metric, all values are normalized by the minimum value among all variants (lower is better for TPOT, higher is better for throughput).



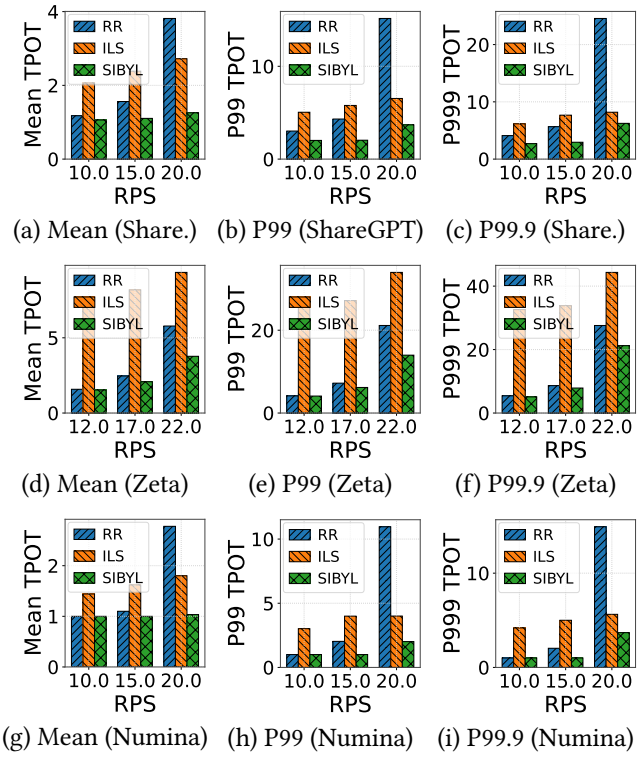
(a) *Mean TPOT* **(b)** *P99 TPOT* **(c)** *P99.9 TPOT* **(d)** *Throughput*

Figure 8. *Sensitivity analysis of Sibyl components on ShareGPT workloads.*

First, Prob@1 disables Sibyl's probabilistic survival weighting by fixing every request's survival probability to 1, i.e., optimistically assuming that all requests will keep running and thus keep contributing to decode load, rather than discounting them by their predicted survival probability. This inflates P99/P99.9 TPOT by 1.26×/1.33× and lowers throughput by 3%, confirming the importance of modeling survival uncertainty for tail latency.

Second, Sibyl estimates decode-phase load from three request categories: ongoing decode requests (Type 1), soon-to-start decode requests (Type 2), and post-handoff lookahead requests (Type 3). Removing Type 1 is the most damaging: P99 TPOT explodes to 12.3× (Mean 8.7×, P99.9 11.8×) and throughput collapses by 38%. Type 1 accounts for the active decoding tokens that constitute the dominant share of an instance's load; blind to it, the scheduler repeatedly routes new requests to already-saturated instances. Type 2 and Type 3 are likewise essential: dropping either still inflates P99.9 TPOT to 1.29× and 1.28× (with a ~3% throughput loss each), confirming that accurately accounting for soon-to-start and post-handoff requests is necessary to suppress worst-case interference. Together, the three categories form a complementary view of decode-phase load, and omitting any one of them measurably degrades tail latency.

Third, ReqLevel replaces our token-based load metric with simple request counts. This degrades P99/P99.9 TPOT



(a) Mean (Share.) (b) P99 (ShareGPT) (c) P99.9 (Share.)

(d) Mean (Zeta) (e) P99 (Zeta) (f) P99.9 (Zeta)

(g) Mean (Numina) (h) P99 (Numina) (i) P99.9 (Numina)

Figure 9. *Simulation on a cluster with 128 decode instances.*

by 1.18×/1.20× and reduces throughput by 2%, demonstrating that counting requests is too coarse to capture the heterogeneous token-level cost of LLM workloads.

6.5 Scalability Performance

To assess Sibyl's scalability in large-scale environments, we implemented a trace-driven discrete-event simulator modeling a cluster with 128 decode instances. The detailed methodology, along with extended evaluations on 256 and 512 instances, is provided in Appendix E.

Performance. Figure 9 shows the performance distribution. Sibyl consistently outperforms baseline strategies across all latency percentiles in large-scale settings. On the heavy-tailed ShareGPT workload, Sibyl improves Mean, P99, and P99.9 TPOT by 33.8%, 55.0%, and 56.4% compared to RR. The advantage over ILS is even more significant, yielding reductions of 52.9% in Mean, 60.7% in P99, and 56.5% in P99.9. The performance gap also shows on the Zeta dataset. Compared to RR, Sibyl reduces Mean, P99, and P99.9 TPOT by 18.1%, 21.9%, and 19.4%, respectively. Against ILS, Sibyl cuts Mean TPOT by 70.4% and suppresses tail latency by 78.0% (P99) and 77.7% (P99.9). The reasoning-driven Numina workload, whose long and highly variable output lengths inflate decode-side contention, exhibits the same trend: Sibyl lowers Mean, P99, and P99.9 TPOT by 19.7%, 42.5%, and 47.7% over RR, and by 37.8%, 71.1%, and 70.1% over ILS, with the tail-latency advantage widening as the output-length variance grows. These results illustrate the scalability bottlenecks of baselines. RR's blind scheduling and ILS's "invisible load" issue lead to significant performance degradation at scale. In contrast, Sibyl's look-ahead scheduling effectively mitigates these inefficiencies.

Dataset	RR	ILS	Sibyl
ShareGPT	53.7%	43.2%	**89.8%** (↑1.67×)
Zeta	36.8%	14.5%	**47.5%** (↑1.29×)
Numina	67.1%	67.4%	**99.9%** (↑1.48×)

Table 4. *Optimal assignment ratio comparison.*

Optimal Assignment Ratio. We further analyze the placement accuracy, defined as the probability that the pre-assigned instance remains the least-loaded node at the actual handoff moment. Table 4 shows that ILS can underperform RR under long-context or highly dynamic workloads (*e.g.,* 14.5% vs. 36.8% on Zeta), indicating that stale load snapshots may cause concurrent requests to collide on the same seemingly idle instance. In contrast, Sibyl consistently improves assignment precision across all datasets, achieving 89.8% on ShareGPT, 47.5% on Zeta, and 99.9% on Numina. Compared with the stronger baseline in each dataset, Sibyl improves the optimal assignment ratio by up to 1.67×, and more than doubles ILS's precision on Zeta. The relatively lower accuracy on Zeta is due to its longer code inputs, which extend the prefill phase, widen the prediction window, and increase prediction uncertainty. This improved assignment precision directly explains the latency reductions observed in Figure 9.

6.6 Overhead Analysis

We analyze the runtime overhead of Sibyl along two dimensions: the scheduling decision latency and the computational resource consumption incurred on the proxy node.

Decision Latency. Table 5 reports the average time to dispatch a single request. Whereas lightweight policies such as RR and ILS make decisions almost instantaneously (< 1 ms), Sibyl incurs an average latency of 5.07 ms, the bulk of which stems from the prefill-duration prediction model used to estimate the handoff moment. Nevertheless, since end-to-end inference latencies for LLM workloads typically span seconds to minutes, this millisecond-level overhead is orders of magnitude smaller and thus negligible to the end-user experience.

Computational Cost. As the scheduling logic for all methods runs on the CPU-based proxy, we track CPU utilization to assess efficiency. As shown in Figure 10, Sibyl yields a utilization profile closely matching the baselines, staying within a low range (2.5%–4.5%) with no noticeable spikes. This confirms that the probabilistic computation and state tracking in Sibyl are lightweight and impose no computational bottleneck on the serving system.

Method	Latency (ms)
RR	<1
ILS	<1
SIBYL	5.07

Table 5. *Comparison of scheduling decision latency.*

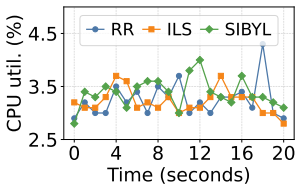


Figure 10. *CPU utilization of the scheduler proxy over time.*

7 Discussion

Learning-based Length Prediction. Recent works explore learning-based approaches that predict generation length to assist request placement [11, 17, 19, 53]. While effective in controlled settings, these designs introduce additional prediction overhead on the scheduling path and require invoking a model for every request. Moreover, learned length prediction faces a robustness–overhead trade-off: higher accuracy typically costs more, while lightweight predictors are less reliable across workloads.

Robustness to Prediction Errors. SIBYL relies on lightweight profiling primitives, including coarse-grained decoding rates (v_k) and survival probabilities ($\mathcal{P}_k$). Importantly, these estimates are not required to be exact: the scheduler only depends on their relative ordering across instances, rather than precise numerical values. As a result, SIBYL can tolerate moderate estimation inaccuracies, as long as the projected load preserves the relative pressure across candidates. This design avoids the need for tightly calibrated predictors on the critical scheduling path.

Limitations and Future Work. (1) The effectiveness of SIBYL is workload-dependent. When decoding is short, when prefill dominates latency, or when decode resources are lightly loaded, the load discrepancy between scheduling and execution is limited, leaving less headroom for projection-based early assignment [9]. (2) SIBYL adopts a per-request greedy strategy to enable fast and scalable scheduling decisions. While global optimization across concurrent arrivals may achieve better assignments under bursty workloads, it would incur higher coordination and computation overhead. (3) SIBYL focuses on mitigating load imbalance at the instance level. It does not explicitly address intra-instance imbalance, such as GPU-level skew introduced by expert parallelism [5, 8, 52, 63]. Incorporating finer-grained, device-level signals into load projection is a promising direction for future work.

8 Related Work

Disaggregated LLM serving. Motivated by the contrasting resource profiles of prefill and decode, disaggregated serving places the two phases on separate instance pools to remove interference and scale each phase independently [7, 18, 31, 35, 36, 46, 65]. Subsequent systems disaggregate additional stages [25, 44], relocate the boundary to trade TTFT for TPOT [14, 27], pool stranded memory [15], make instances stateless to flip roles under bursty traffic [38, 56], separate prefill and decode *within* a single GPU [42], or unify aggregated and disaggregated execution [58]. Across this spectrum, the architecture defines *where* the phases execute, yet shares a common premise: a request is bound to its decode instance at prefill start so that KV-cache transfer overlaps with computation [35, 36]. SIBYL is orthogonal: it leaves the architecture untouched and supplies the missing primitive of *which* decode instance to bind to, dropping in as a black-box proxy for any such system.

Predictive and length-aware scheduling. Since output length is unknown a priori, engines often fall back on first-come-first-serve and suffer head-of-line blocking. A growing line predicts request lengths and orders execution accordingly: S^3 [19] packs batches from predicted lengths, learning-to-rank approximates shortest-job-first from predicted output ranks [11], and embedding-based [40], iterative [3], and semi-clairvoyant [22] predictors drive shortest-remaining-time scheduling within a single engine. Yet exact per-request prediction is infeasible [11] and brittle under the heavy-tailed outputs of reasoning workloads (§2.2), and these methods target intra-instance *ordering* rather than placement. SIBYL instead decides cross-instance *placement* without point estimates: it models active-task *residual lifetimes* via survival analysis and incorporates shadow load from in-flight prefills, yielding a probabilistic occupancy projection.

Capacity provisioning and autoscaling. A complementary direction adapts the *amount* of provisioned capacity to demand: autoscaling that cuts scaling latency via live layer-granular scaling over the compute network [61] or low-cold-start serverless inference [10], energy co-optimization [45], coordination of heterogeneous disaggregated pools [23], and cross-model GPU pooling [57]. Closest to us, Aladdin [30] jointly scales resources and places requests by bin-packing on predicted lengths. These mechanisms reconfigure the cluster over coarse timescales; SIBYL is orthogonal and operates at per-request granularity on a *fixed* pool, complementing such elasticity after capacity provisioning.

9 Conclusion

We identify a fundamental *load discrepancy* in disaggregated LLM serving, where scheduling decisions made at prefill start often become stale by the time decoding begins. To address this issue, we present SIBYL, a speculative scheduler that projects per-instance load at the handoff moment using lightweight survival-based primitives. This projection guides load-aware early assignment and helps align early assignment with actual decode execution. Experiments on production traces and large-scale simulations show that SIBYL reduces tail latency by up to 56.9% compared to state-of-the-art schedulers.

References

[1] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S Gulavani, Alexey Tumanov, and Ramachandran Ramjee. 2024. Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve. *Proceedings of 18th USENIX Symposium on Operating Systems Design and Implementation, 2024, Santa Clara* (2024).

[2] AI@Meta. 2024. Llama 3 Model Card. (2024). https://github.com/meta-llama/llama3/blob/main/MODEL_CARD.md

[3] Seungbeom Choi, Jeonghoe Goo, Eunjoo Jeon, Mingyu Yang, and Minsung Jang. 2025. ELIS: Efficient LLM Iterative Scheduling System with Response Length Predictor. arXiv:2505.09142 [cs.DC] https://arxiv.org/abs/2505.09142

[4] Gheorghe Comanici, Eric Bieber, Mike Schaekermann, Ice Pasupat, Noveen Sachdeva, Inderjit Dhillon, Marcel Blistein, Ori Ram, Dan Zhang, Evan Rosen, et al. 2025. Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities. *arXiv preprint arXiv:2507.06261* (2025).

[5] DeepSeek-AI. 2024. EPLB: Expert Parallel Load Balancing. https://github.com/deepseek-ai/EPLB. GitHub repository.

[6] DeepSeek-AI. 2026. DeepSeek-V4: Towards Highly Efficient Million-Token Context Intelligence.

[7] DeepSeek-AI, Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, Damai Dai, Daya Guo, Dejian Yang, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Han Bao, Hanwei Xu, Haocheng Wang, Haowei Zhang, Honghui Ding, Huajian Xin, Huazuo Gao, Hui Li, Hui Qu, J. L. Cai, Jian Liang, Jianzhong Guo, Jiaqi Ni, Jiashi Li, Jiawei Wang, Jin Chen, Jingchang Chen, Jingyang Yuan, Junjie Qiu, Junlong Li, Junxiao Song, Kai Dong, Kai Hu, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Lei Xu, Leyi Xia, Liang Zhao, Litong Wang, Liyue Zhang, Meng Li, Miaojun Wang, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Mingming Li, Ning Tian, Panpan Huang, Peiyi Wang, Peng Zhang, Qiancheng Wang, Qihao Zhu, Qinyu Chen, Qiushi Du, R. J. Chen, R. L. Jin, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, Runxin Xu, Ruoyu Zhang, Ruyi Chen, S. S. Li, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shaoqing Wu, Shengfeng Ye, Shengfeng Ye, Shirong Ma, Shiyu Wang, Shuang Zhou, Shuiping Yu, Shunfeng Zhou, Shuting Pan, T. Wang, Tao Yun, Tian Pei, Tianyu Sun, W. L. Xiao, Wangding Zeng, Wanjia Zhao, Wei An, Wen Liu, Wenfeng Liang, Wenjun Gao, Wenqin Yu, Wentao Zhang, X. Q. Li, Xiangyue Jin, Xianzu Wang, Xiao Bi, Xiaodong Liu, Xiaohan Wang, Xiaojin Shen, Xiaokang Chen, Xiaokang Zhang, Xiaosha Chen, Xiaotao Nie, Xiaowen Sun, Xiaoxiang Wang, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xingkai Yu, Xinnan Song, Xinxia Shan, Xinyi Zhou, Xinyu Yang, Xinyuan Li, Xuecheng Su, Xuheng Lin, Y. K. Li, Y. Q. Wang, Y. X. Wei, Y. X. Zhu, Yang Zhang, Yanhong Xu, Yanhong Xu, Yanping Huang, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Li, Yaohui Wang, Yi Yu, Yi Zheng, Yichao Zhang, Yifan Shi, Yiliang Xiong, Ying He, Ying Tang, Yishi Piao, Yisong Wang, Yixuan Tan, Yiyang Ma, Yiyuan Liu, Yongqiang Guo, Yu Wu, Yuan Ou, Yuchen Zhu, Yuduan Wang, Yue Gong, Yuheng Zou, Yujia He, Yukun Zha, Yunfan Xiong, Yunxian Ma, Yuting Yan, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuyang Zhou, Z. F. Wu, Z. Z. Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhen Huang, Zhen Zhang, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhibin Gou, Zhicheng Ma, Zhigang Yan, Zhihong Shao, Zhipeng Xu, Zhiyu Wu, Zhongyu Zhang, Zhuoshu Li, Zihui Gu, Zijia Zhu, Zijun Liu, Zilin Li, Ziwei Xie, Ziyang Song, Ziyi Gao, and Zizheng Pan. 2025. DeepSeek-V3 Technical Report. arXiv:2412.19437 [cs.CL] https://arxiv.org/abs/2412.19437

[8] Zachary Doucet, Rishi Sharma, Martijn de Vos, Rafael Pires, Anne-Marie Kermarrec, and Oana Balmau. 2025. HarMoEny: Efficient Multi-GPU Inference of MoE Models. arXiv:2506.12417 [cs.DC] https://arxiv.org/abs/2506.12417

[9] Kuntai Du, Bowen Wang, Chen Zhang, Yiming Cheng, Qing Lan, Hejian Sang, Yihua Cheng, Jiayi Yao, Xiaoxuan Liu, Yifan Qiao, Ion Stoica, and Junchen Jiang. 2025. PrefillOnly: An Inference Engine for Prefill-only Workloads in Large Language Model Applications. arXiv:2505.07203 [cs.DC] https://arxiv.org/abs/2505.07203

[10] Yao Fu, Leyang Xue, Yeqi Huang, Andrei-Octavian Brabete, Dmitrii Ustiugov, Yuvraj Patel, and Luo Mai. 2024. ServerlessLLM: Low-Latency Serverless Inference for Large Language Models. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. USENIX Association, 135–153. https://www.usenix.org/conference/osdi24/presentation/fu

[11] Yichao Fu, Siqi Zhu, Runlong Su, Aurick Qiao, Ion Stoica, and Hao Zhang. 2024. Efficient LLM Scheduling by Learning to Rank. arXiv:2408.15792 [cs.LG] https://arxiv.org/abs/2408.15792

[12] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, Amy Yang, Angela Fan, Anirudh Goyal, Anthony Hartshorn, Aobo Yang, Archi Mitra, Archie Sravankumar, Artem Korenev, Arthur Hinsvark, Arun Rao, Aston Zhang, Aurelien Rodriguez, Austen Gregerson, Ava Spataru, Baptiste Roziere, Bethany Biron, Binh Tang, Bobbie Chern, Charlotte Caucheteux, Chaya Nayak, Chloe Bi, Chris Marra, Chris McConnell, Christian Keller, Christophe Touret, Chunyang Wu, Corinne Wong, Cristian Canton Ferrer, Cyrus Nikolaidis, Damien Allonsius, Daniel Song, Danielle Pintz, Danny Livshits, Danny Wyatt, David Esiobu, Dhruv Choudhary, Dhruv Mahajan, Diego Garcia-Olano, Diego Perino, Dieuwke Hupkes, Egor Lakomkin, Ehab AlBadawy, Elina Lobanova, Emily Dinan, Eric Michael Smith, Filip Radenovic, Francisco Guzmán, Frank Zhang, Gabriel Synnaeve, Gabrielle Lee, Georgia Lewis Anderson, Govind Thattai, Graeme Nail, Gregoire Mialon, Guan Pang, Guillem Cucurell, Hailey Nguyen, Hannah Korevaar, Hu Xu, Hugo Touvron, Iliyan Zarov, Imanol Arrieta Ibarra, Isabel Kloumann, Ishan Misra, Ivan Evtimov, Jack Zhang, Jade Copet, Jaewon Lee, Jan Geffert, Jana Vranes, Jason Park, Jay Mahadeokar, Jeet Shah, Jelmer van der Linde, Jennifer Billock, Jenny Hong, Jenya Lee, Jeremy Fu, Jianfeng Chi, Jianyu Huang, Jiawen Liu, Jie Wang, Jiecao Yu, Joanna Bitton, Joe Spisak, Jongsoo Park, Joseph Rocca, Joshua Johnstun, Joshua Saxe, Junteng Jia, Kalyan Vasuden Alwala, Karthik Prasad, Kartikeya Upasani, Kate Plawiak, Ke Li, Kenneth Heafield, Kevin Stone, Khalid El-Arini, Krithika Iyer, Kshitiz Malik, Kuenley Chiu, Kunal Bhalla, Kushal Lakhotia, Lauren Rantala-Yeary, Laurens van der Maaten, Lawrence Chen, Liang Tan, Liz Jenkins, Louis Martin, Lovish Madaan, Lubo Malo, Lukas Blecher, Lukas Landzaat, Luke de Oliveira, Madeline Muzzi, Mahesh Pasupuleti, Mannat Singh, Manohar Paluri, Marcin Kardas, Maria Tsimpoukelli, Mathew Oldham, Mathieu Rita, Maya Pavlova, Melanie Kambadur, Mike Lewis, Min Si, Mitesh Kumar Singh, Mona Hassan, Naman Goyal, Narjes Torabi, Nikolay Bashlykov, Nikolay Bogoychev, Niladri Chatterji, Ning Zhang, Olivier Duchenne, Onur Çelebi, Patrick Alrassy, Pengchuan Zhang, Pengwei Li, Petar Vasic, Peter Weng, Prajjwal Bhargava, Pratik Dubal, Praveen Krishnan, Punit Singh Koura, Puxin Xu, Qing He, Qingxiao Dong, Ragavan Srinivasan, Raj Ganapathy, Ramon Calderer, Ricardo Silveira Cabral, Robert Stojnic, Roberta Raileanu, Rohan Maheswari, Rohit Girdhar, Rohit Patel, Romain Sauvestre, Ronnie Polidoro, Roshan Sumbaly, Ross Taylor, Ruan Silva, Rui Hou, Rui Wang, Saghar Hosseini, Sahana Chennabasappa, Sanjay Singh, Sean Bell, Seohyun Sonia Kim, Sergey Edunov, Shaoliang Nie, Sharan Narang, Sharath Raparthy, Sheng Shen, Shengye Wan, Shruti Bhosale, Shun Zhang, Simon Vandenhende, Soumya Batra, Spencer Whitman, Sten Sootla, Stephane Collot, Suchin Gururangan, Sydney Borodinsky, Tamar Herman, Tara Fowler, Tarek Sheasha, Thomas Georgiou, Thomas Scialom, Tobias Speckbacher, Todor Mihaylov, Tong Xiao, Ujjwal Karn, Vedanuj Goswami, Vibhor Gupta, Vignesh Ramanathan, Viktor Kerkez, Vincent Gonguet, Virginie Do, Vish Vogeti, Vítor Albiero, Vladan Petrovic, Weiwei Chu, Wenhan Xiong, Wenyin Fu, Whitney Meers, Xavier Martinet,

Xiaodong Wang, Xiaofang Wang, Xiaoqing Ellen Tan, Xide Xia, Xinfeng Xie, Xuchao Jia, Xuewei Wang, Yaelle Goldschlag, Yashesh Gaur, Yasmine Babaei, Yi Wen, Yiwen Song, Yuchen Zhang, Yue Li, Yuning Mao, Zacharie Delpierre Coudert, Zheng Yan, Zhengxing Chen, Zoe Papakipos, Aaditya Singh, Aayushi Srivastava, Abha Jain, Adam Kelsey, Adam Shajnfeld, Adithya Gangidi, Adolfo Victoria, Ahuva Goldstand, Ajay Menon, Ajay Sharma, Alex Boesenberg, Alexei Baevski, Allie Feinstein, Amanda Kallet, Amit Sangani, Amos Teo, Anam Yunus, Andrei Lupu, Andres Alvarado, Andrew Caples, Andrew Gu, Andrew Ho, Andrew Poulton, Andrew Ryan, Ankit Ramchandani, Annie Dong, Annie Franco, Anuj Goyal, Aparajita Saraf, Arkabandhu Chowdhury, Ashley Gabriel, Ashwin Bharambe, Assaf Eisenman, Azadeh Yazdan, Beau James, Ben Maurer, Benjamin Leonhardi, Bernie Huang, Beth Loyd, Beto De Paola, Bhargavi Paranjape, Bing Liu, Bo Wu, Boyu Ni, Braden Hancock, Bram Wasti, Brandon Spence, Brani Stojkovic, Brian Gamido, Britt Montalvo, Carl Parker, Carly Burton, Catalina Mejia, Ce Liu, Changhan Wang, Changkyu Kim, Chao Zhou, Chester Hu, Ching-Hsiang Chu, Chris Cai, Chris Tindal, Christoph Feichtenhofer, Cynthia Gao, Damon Civin, Dana Beaty, Daniel Kreymer, Daniel Li, David Adkins, David Xu, Davide Testuggine, Delia David, Devi Parikh, Diana Liskovich, Didem Foss, Dingkang Wang, Duc Le, Dustin Holland, Edward Dowling, Eissa Jamil, Elaine Montgomery, Eleonora Presani, Emily Hahn, Emily Wood, Eric-Tuan Le, Erik Brinkman, Esteban Arcaute, Evan Dunbar, Evan Smothers, Fei Sun, Felix Kreuk, Feng Tian, Filippos Kokkinos, Firat Ozgenel, Francesco Caggioni, Frank Kanayet, Frank Seide, Gabriela Medina Florez, Gabriella Schwarz, Gada Badeer, Georgia Swee, Gil Halpern, Grant Herman, Grigory Sizov, Guangyi, Zhang, Guna Lakshminarayanan, Hakan Inan, Hamid Shojanazeri, Han Zou, Hannah Wang, Hanwen Zha, Haroun Habeeb, Harrison Rudolph, Helen Suk, Henry Aspegren, Hunter Goldman, Hongyuan Zhan, Ibrahim Damlaj, Igor Molybog, Igor Tufanov, Ilias Leontiadis, Irina-Elena Veliche, Itai Gat, Jake Weissman, James Geboski, James Kohli, Janice Lam, Japhet Asher, Jean-Baptiste Gaya, Jeff Marcus, Jeff Tang, Jennifer Chan, Jenny Zhen, Jeremy Reizenstein, Jeremy Teboul, Jessica Zhong, Jian Jin, Jingyi Yang, Joe Cummings, Jon Carvill, Jon Shepard, Jonathan McPhie, Jonathan Torres, Josh Ginsburg, Junjie Wang, Kai Wu, Kam Hou U, Karan Saxena, Kartikay Khandelwal, Katayoun Zand, Kathy Matosich, Kaushik Veeraraghavan, Kelly Michelena, Keqian Li, Kiran Jagadeesh, Kun Huang, Kunal Chawla, Kyle Huang, Lailin Chen, Lakshya Garg, Lavender A, Leandro Silva, Lee Bell, Lei Zhang, Liangpeng Guo, Licheng Yu, Liron Moshkovich, Luca Wehrstedt, Madian Khabsa, Manav Avalani, Manish Bhatt, Martynas Mankus, Matan Hasson, Matthew Lennie, Matthias Reso, Maxim Groshev, Maxim Naumov, Maya Lathi, Meghan Keneally, Miao Liu, Michael L. Seltzer, Michal Valko, Michelle Restrepo, Mihir Patel, Mik Vyatskov, Mikayel Samvelyan, Mike Clark, Mike Macey, Mike Wang, Miquel Jubert Hermoso, Mo Metanat, Mohammad Rastegari, Munish Bansal, Nandhini Santhanam, Natascha Parks, Natasha White, Navyata Bawa, Nayan Singhal, Nick Egebo, Nicolas Usunier, Nikhil Mehta, Nikolay Pavlovich Laptev, Ning Dong, Norman Cheng, Oleg Chernoguz, Olivia Hart, Omkar Salpekar, Ozlem Kalinli, Parkin Kent, Parth Parekh, Paul Saab, Pavan Balaji, Pedro Rittner, Philip Bontrager, Pierre Roux, Piotr Dollar, Polina Zvyagina, Prashant Ratanchandani, Pritish Yuvraj, Qian Liang, Rachad Alao, Rachel Rodriguez, Rafi Ayub, Raghotham Murthy, Raghu Nayani, Rahul Mitra, Rangaprabhu Parthasarathy, Raymond Li, Rebekkah Hogan, Robin Battey, Rocky Wang, Russ Howes, Ruty Rinott, Sachin Mehta, Sachin Siby, Sai Jayesh Bondu, Samyak Datta, Sara Chugh, Sara Hunt, Sargun Dhillon, Sasha Sidorov, Satadru Pan, Saurabh Mahajan, Saurabh Verma, Seiji Yamamoto, Sharadh Ramaswamy, Shaun Lindsay, Shaun Lindsay, Sheng Feng, Shenghao Lin, Shengxin Cindy Zha, Shishir Patil, Shiva Shankar, Shuqiang Zhang, Shuqiang Zhang, Sinong Wang, Sneha Agarwal, Soji Sajuyigbe, Soumith Chintala, Stephanie Max, Stephen Chen, Steve Kehoe, Steve Satterfield, Sudarshan Govindaprasad, Sumit Gupta, Summer Deng, Sungmin Cho, Sunny Virk, Suraj Subramanian, Sy Choudhury, Sydney Goldman, Tal Remez, Tamar Glaser, Tamara Best, Thilo Koehler, Thomas Robinson, Tianhe Li, Tianjun Zhang, Tim Matthews, Timothy Chou, Tzook Shaked, Varun Vontimitta, Victoria Ajayi, Victoria Montanez, Vijai Mohan, Vinay Satish Kumar, Vishal Mangla, Vlad Ionescu, Vlad Poenaru, Vlad Tiberiu Mihailescu, Vladimir Ivanov, Wei Li, Wenchen Wang, Wenwen Jiang, Wes Bouaziz, Will Constable, Xiaocheng Tang, Xiaojian Wu, Xiaolan Wang, Xilun Wu, Xinbo Gao, Yaniv Kleinman, Yanjun Chen, Ye Hu, Ye Jia, Ye Qi, Yenda Li, Yilin Zhang, Ying Zhang, Yossi Adi, Youngjin Nam, Yu, Wang, Yu Zhao, Yuchen Hao, Yundi Qian, Yunlu Li, Yuzi He, Zach Rait, Zachary DeVito, Zef Rosnbrick, Zhaoduo Wen, Zhenyu Yang, Zhiwei Zhao, and Zhiyu Ma. 2024. The Llama 3 Herd of Models. arXiv:2407.21783 [cs.AI] https://arxiv.org/abs/2407.21783

[13] Morgan Heisler, Zahra Yousefijamarani, Xinglu Wang, Qian Wang, Ge Shi, Hanieh Sadri, Timothy Yu, Yifei Li, Haley Li, Gursimran Singh, et al. 2025. LLM Inference Scheduling: A Survey of Techniques, Frameworks, and Trade-offs. *Authorea Preprints* (2025).

[14] Ke Hong, Lufang Chen, Zhong Wang, Xiuhong Li, Qiuli Mao, Jianping Ma, Chao Xiong, Guanyu Wu, Buhe Han, Guohao Dai, Yun Liang, and Yu Wang. 2025. semi-PD: Towards Efficient LLM Serving via Phase-Wise Disaggregated Computation and Unified Storage. arXiv:2504.19867 [cs.CL] https://arxiv.org/abs/2504.19867

[15] Cunchen Hu, Heyang Huang, Junhao Hu, Jiang Xu, Xusheng Chen, Tao Xie, Chenxi Wang, Sa Wang, Yungang Bao, Ninghui Sun, and Yizhou Shan. 2024. MemServe: Context Caching for Disaggregated LLM Serving with Elastic Memory Pool. arXiv:2406.17565 [cs.DC] https://arxiv.org/abs/2406.17565

[16] Kunal Jain, Anjaly Parayil, Ankur Mallick, Esha Choukse, Xiaoting Qin, Jue Zhang, Íñigo Goiri, Rujia Wang, Chetan Bansal, Victor Rühle, Anoop Kulkarni, Steve Kofsky, and Saravan Rajmohan. 2025. Performance Aware LLM Load Balancer for Mixed Workloads. In *Proceedings of the 5th Workshop on Machine Learning and Systems* (World Trade Center, Rotterdam, Netherlands) *(EuroMLSys '25)*. Association for Computing Machinery, New York, NY, USA, 19–30. https://doi.org/10.1145/3721146.3721947

[17] Kunal Jain, Anjaly Parayil, Ankur Mallick, Esha Choukse, Xiaoting Qin, Jue Zhang, Íñigo Goiri, Rujia Wang, Chetan Bansal, Victor Rühle, Anoop Kulkarni, Steve Kofsky, and Saravan Rajmohan. 2025. Intelligent Router for LLM Workloads: Improving Performance Through Workload-Aware Load Balancing. arXiv:2408.13510 [cs.DC] https://arxiv.org/abs/2408.13510

[18] Yibo Jin, Tao Wang, Huimin Lin, Mingyang Song, Peiyang Li, Yipeng Ma, Yicheng Shan, Zhengfan Yuan, Cailong Li, Yajing Sun, Tiandeng Wu, Xing Chu, Ruizhi Huan, Li Ma, Xiao You, Wenting Zhou, Yunpeng Ye, Wen Liu, Xiangkun Xu, Yongsheng Zhang, Tiantian Dong, Jiawei Zhu, Zhe Wang, Xijian Ju, Jianxun Song, Haoliang Cheng, Xiaojing Li, Jiandong Ding, Hefei Guo, and Zhengyong Zhang. 2024. P/D-Serve: Serving Disaggregated Large Language Model at Scale. arXiv:2408.08147 [cs.DC] https://arxiv.org/abs/2408.08147

[19] Yunho Jin, Chun-Feng Wu, David Brooks, and Gu-Yeon Wei. 2023. S^3: Increasing GPU Utilization during Generative Inference for Higher Throughput. arXiv:2306.06000 [cs.AR] https://arxiv.org/abs/2306.06000

[20] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*.

[21] Jia LI, Edward Beeching, Lewis Tunstall, Ben Lipkin, Roman Soletskyi, Shengyi Costa Huang, Kashif Rasul, Longhui Yu, Albert Jiang, Ziju Shen, Zihan Qin, Bin Dong, Li Zhou, Yann Fleureau, Guillaume Lample, and Stanislas Polu. 2024. NuminaMath. https://huggingface.co/datasets/AI-MO/NuminaMath-1.5.

[22] Ruixiao Li, Fahao Chen, and Peng Li. 2025. Semi-Clairvoyant Scheduling of Speculative Decoding Requests to Minimize LLM Inference Latency. arXiv:2505.17074 [cs.CL] https://arxiv.org/abs/2505.17074

[23] Rongzhi Li, Ruogu Du, Zefang Chu, Sida Zhao, Chunlei Han, Zuocheng Shi, Yiwen Shao, Huanle Han, Long Huang, Zherui Liu, and Shufan Liu. 2025. Taming the Chaos: Coordinated Autoscaling for Heterogeneous and Disaggregated LLM Inference. arXiv:2508.19559 [cs.DC] https://arxiv.org/abs/2508.19559

[24] Weiqing Li, Guochao Jiang, Xiangyong Ding, Zhangcheng Tao, Chuzhan Hao, Chenfeng Xu, Yuewei Zhang, and Hao Wang. 2025. FlowKV: A Disaggregated Inference Framework with Low-Latency KV Cache Transfer and Load-Aware Scheduling. arXiv:2504.03775 [cs.DC] https://arxiv.org/abs/2504.03775

[25] Yunkai Liang, Zhangyu Chen, Pengfei Zuo, Zhi Zhou, Xu Chen, and Zhou Yu. 2025. Injecting Adrenaline into LLM Serving: Boosting Resource Utilization and Throughput via Attention Disaggregation. arXiv:2503.20552 [cs.DC] https://arxiv.org/abs/2503.20552

[26] Chiheng Lou, Sheng Qi, Chao Jin, Dapeng Nie, Haoran Yang, Yu Ding, Xuanzhe Liu, and Xin Jin. 2025. HydraServe: Minimizing Cold Start Latency for Serverless LLM Serving in Public Clouds. arXiv:2502.15524 [cs.DC] https://arxiv.org/abs/2502.15524

[27] Jinming Ma, Jiefei Chen, Xiuhong Li, Jiangfei Duan, Haojie Duanmu, Xingcheng Zhang, Chao Yang, and Dahua Lin. 2025. *Tropical: Enhancing SLO Attainment in Disaggregated LLM Serving via SLO-Aware Multiplexing*. IEEE Press. https://doi.org/10.1109/DAC63849.2025.11132617

[28] Shervin Minaee, Tomas Mikolov, Narjes Nikzad, Meysam Chenaghlu, Richard Socher, Xavier Amatriain, and Jianfeng Gao. 2025. Large Language Models: A Survey. arXiv:2402.06196 [cs.CL] https://arxiv.org/abs/2402.06196

[29] MiniMax, Aonian Li, Bangwei Gong, Bo Yang, Boji Shan, Chang Liu, Cheng Zhu, Chunhao Zhang, Congchao Guo, Da Chen, Dong Li, Enwei Jiao, Gengxin Li, Guojun Zhang, Haohai Sun, Houze Dong, Jiadai Zhu, Jiaqi Zhuang, Jiayuan Song, Jin Zhu, Jingtao Han, Jingyang Li, Junbin Xie, Junhao Xu, Junjie Yan, Kaishun Zhang, Kecheng Xiao, Kexi Kang, Le Han, Leyang Wang, Lianfei Yu, Liheng Feng, Lin Zheng, Linbo Chai, Long Xing, Meizhi Ju, Mingyuan Chi, Mozhi Zhang, Peikai Huang, Pengcheng Niu, Pengfei Li, Pengyu Zhao, Qi Yang, Qidi Xu, Qiexiang Wang, Qin Wang, Qiuhui Li, Ruitao Leng, Shengmin Shi, Shuqi Yu, Sichen Li, Songquan Zhu, Tao Huang, Tianrun Liang, Weigao Sun, Weixuan Sun, Weiyu Cheng, Wenkai Li, Xiangjun Song, Xiao Su, Xiaodong Han, Xinjie Zhang, Xinzhu Hou, Xu Min, Xun Zou, Xuyang Shen, Yan Gong, Yingjie Zhu, Yipeng Zhou, Yiran Zhong, Yongyi Hu, Yuanxiang Fan, Yue Yu, Yufeng Yang, Yuhao Li, Yunan Huang, Yunji Li, Yunpeng Huang, Yunzhi Xu, Yuxin Mao, Zehan Li, Zekang Li, Zewei Tao, Zewen Ying, Zhaoyang Cong, Zhen Qin, Zhenhua Fan, Zhihang Yu, Zhuo Jiang, and Zijia Wu. 2025. MiniMax-01: Scaling Foundation Models with Lightning Attention. arXiv:2501.08313 [cs.CL] https://arxiv.org/abs/2501.08313

[30] Chengyi Nie, Rodrigo Fonseca, and Zhenhua Liu. 2024. Aladdin: Joint Placement and Scaling for SLO-Aware LLM Serving. arXiv:2405.06856 [cs.DC] https://arxiv.org/abs/2405.06856

[31] NVIDIA. 2025. Dynamo: Designing LLM Inference Clusters for Performance and Energy Efficiency. https://github.com/ai-dynamo/dynamo.

[32] NVIDIA Corporation. 2025. NVIDIA Collective Communication Library (NCCL). https://github.com/NVIDIA/nccl.

[33] OpenAI, :, Aaron Jaech, Adam Kalai, Adam Lerer, Adam Richardson, Ahmed El-Kishky, Aiden Low, Alec Helyar, Aleksander Madry, Alex Beutel, Alex Carney, Alex Iftimie, Alex Karpenko, Alex Tachard Passos, Alexander Neitz, Alexander Prokofiev, Alexander Wei, Allison Tam, Ally Bennett, Ananya Kumar, Andre Saraiva, Andrea Vallone, Andrew Duberstein, Andrew Kondrich, Andrey Mishchenko, Andy Applebaum, Angela Jiang, Ashvin Nair, Barret Zoph, Behrooz Ghorbani, Ben Rossen, Benjamin Sokolowsky, Boaz Barak, Bob McGrew, Borys Minaiev, Botao Hao, Bowen Baker, Brandon Houghton, Brandon McKinzie, Brydon Eastman, Camillo Lugaresi, Cary Bassin, Cary Hudson, Chak Ming Li, Charles de Bourcy, Chelsea Voss, Chen Shen, Chong Zhang, Chris Koch, Chris Orsinger, Christopher Hesse, Claudia Fischer, Clive Chan, Dan Roberts, Daniel Kappler, Daniel Levy, Daniel Selsam, David Dohan, David Farhi, David Mely, David Robinson, Dimitris Tsipras, Doug Li, Dragos Oprica, Eben Freeman, Eddie Zhang, Edmund Wong, Elizabeth Proehl, Enoch Cheung, Eric Mitchell, Eric Wallace, Erik Ritter, Evan Mays, Fan Wang, Felipe Petroski Such, Filippo Raso, Florencia Leoni, Foivos Tsimpourlas, Francis Song, Fred von Lohmann, Freddie Sulit, Geoff Salmon, Giambattista Parascandolo, Gildas Chabot, Grace Zhao, Greg Brockman, Guillaume Leclerc, Hadi Salman, Haiming Bao, Hao Sheng, Hart Andrin, Hessam Bagherinezhad, Hongyu Ren, Hunter Lightman, Hyung Won Chung, Ian Kivlichan, Ian O'Connell, Ian Osband, Ignasi Clavera Gilaberte, Ilge Akkaya, Ilya Kostrikov, Ilya Sutskever, Irina Kofman, Jakub Pachocki, James Lennon, Jason Wei, Jean Harb, Jerry Tworek, Jiacheng Feng, Jiahui Yu, Jiayi Weng, Jie Tang, Jieqi Yu, Joaquin Quiñonero Candela, Joe Palermo, Joel Parish, Johannes Heidecke, John Hallman, John Rizzo, Jonathan Gordon, Jonathan Uesato, Jonathan Ward, Joost Huizinga, Julie Wang, Kai Chen, Kai Xiao, Karan Singhal, Karina Nguyen, Karl Cobbe, Katy Shi, Kayla Wood, Kendra Rimbach, Keren Gu-Lemberg, Kevin Liu, Kevin Lu, Kevin Stone, Kevin Yu, Lama Ahmad, Lauren Yang, Leo Liu, Leon Maksin, Leyton Ho, Liam Fedus, Lilian Weng, Linden Li, Lindsay McCallum, Lindsey Held, Lorenz Kuhn, Lukas Kondraciuk, Lukasz Kaiser, Luke Metz, Madelaine Boyd, Maja Trebacz, Manas Joglekar, Mark Chen, Marko Tintor, Mason Meyer, Matt Jones, Matt Kaufer, Max Schwarzer, Meghan Shah, Mehmet Yatbaz, Melody Y. Guan, Mengyuan Xu, Mengyuan Yan, Mia Glaese, Mianna Chen, Michael Lampe, Michael Malek, Michele Wang, Michelle Fradin, Mike McClay, Mikhail Pavlov, Miles Wang, Mingxuan Wang, Mira Murati, Mo Bavarian, Mostafa Rohaninejad, Nat McAleese, Neil Chowdhury, Neil Chowdhury, Nick Ryder, Nikolas Tezak, Noam Brown, Ofir Nachum, Oleg Boiko, Oleg Murk, Olivia Watkins, Patrick Chao, Paul Ashbourne, Pavel Izmailov, Peter Zhokhov, Rachel Dias, Rahul Arora, Randall Lin, Rapha Gontijo Lopes, Raz Gaon, Reah Miyara, Reimar Leike, Renny Hwang, Rhythm Garg, Robin Brown, Roshan James, Rui Shu, Ryan Cheu, Ryan Greene, Saachi Jain, Sam Altman, Sam Toizer, Sam Toyer, Samuel Miserendino, Sandhini Agarwal, Santiago Hernandez, Sasha Baker, Scott McKinney, Scottie Yan, Shengjia Zhao, Shengli Hu, Shibani Santurkar, Shraman Ray Chaudhuri, Shuyuan Zhang, Siyuan Fu, Spencer Papay, Steph Lin, Suchir Balaji, Suvansh Sanjeev, Szymon Sidor, Tal Broda, Aidan Clark, Tao Wang, Taylor Gordon, Ted Sanders, Tejal Patwardhan, Thibault Sottiaux, Thomas Degry, Thomas Dimson, Tianhao Zheng, Timur Garipov, Tom Stasi, Trapit Bansal, Trevor Creech, Troy Peterson, Tyna Eloundou, Valerie Qi, Vineet Kosaraju, Vinnie Monaco, Vitchyr Pong, Vlad Fomenko, Weiyi Zheng, Wenda Zhou, Wes McCabe, Wojciech Zaremba, Yann Dubois, Yinghai Lu, Yining Chen, Young Cha, Yu Bai, Yuchen He, Yuchen Zhang, Yunyun Wang, Zheng Shao, and Zhuohan Li. 2024. OpenAI o1 System Card. arXiv:2412.16720 [cs.AI] https://arxiv.org/abs/2412.16720

[34] OpenAI, Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, Red Avila, Igor Babuschkin, Suchir Balaji, Valerie Balcom, Paul Baltescu, Haiming Bao, Mohammad Bavarian, Jeff Belgum, Irwan Bello, Jake Berdine, Gabriel Bernadett-Shapiro, Christopher Berner, Lenny Bogdonoff, Oleg Boiko, Madelaine Boyd, Anna-Luisa Brakman, Greg Brockman, Tim Brooks, Miles Brundage, Kevin Button, Trevor Cai, Rosie Campbell, Andrew Cann, Brittany Carey, Chelsea Carlson, Rory Carmichael, Brooke Chan, Che Chang, Fotis Chantzis, Derek Chen, Sully Chen, Ruby Chen, Jason Chen, Mark Chen, Ben Chess, Chester Cho, Casey Chu, Hyung Won Chung, Dave Cummings, Jeremiah Currier, Yunxing Dai, Cory Decareaux, Thomas Degry, Noah Deutsch, Damien

Deville, Arka Dhar, David Dohan, Steve Dowling, Sheila Dunning, Adrien Ecoffet, Atty Eleti, Tyna Eloundou, David Farhi, Liam Fedus, Niko Felix, Simón Posada Fishman, Juston Forte, Isabella Fulford, Leo Gao, Elie Georges, Christian Gibson, Vik Goel, Tarun Gogineni, Gabriel Goh, Rapha Gontijo-Lopes, Jonathan Gordon, Morgan Grafstein, Scott Gray, Ryan Greene, Joshua Gross, Shixiang Shane Gu, Yufei Guo, Chris Hallacy, Jesse Han, Jeff Harris, Yuchen He, Mike Heaton, Johannes Heidecke, Chris Hesse, Alan Hickey, Wade Hickey, Peter Hoeschele, Brandon Houghton, Kenny Hsu, Shengli Hu, Xin Hu, Joost Huizinga, Shantanu Jain, Shawn Jain, Joanne Jang, Angela Jiang, Roger Jiang, Haozhun Jin, Denny Jin, Shino Jomoto, Billie Jonn, Heewoo Jun, Tomer Kaftan, Łukasz Kaiser, Ali Kamali, Ingmar Kanitscheider, Nitish Shirish Keskar, Tabarak Khan, Logan Kilpatrick, Jong Wook Kim, Christina Kim, Yongjik Kim, Jan Hendrik Kirchner, Jamie Kiros, Matt Knight, Daniel Kokotajlo, Łukasz Kondraciuk, Andrew Kondrich, Aris Konstantinidis, Kyle Kosic, Gretchen Krueger, Vishal Kuo, Michael Lampe, Ikai Lan, Teddy Lee, Jan Leike, Jade Leung, Daniel Levy, Chak Ming Li, Rachel Lim, Molly Lin, Stephanie Lin, Mateusz Litwin, Theresa Lopez, Ryan Lowe, Patricia Lue, Anna Makanju, Kim Malfacini, Sam Manning, Todor Markov, Yaniv Markovski, Bianca Martin, Katie Mayer, Andrew Mayne, Bob McGrew, Scott Mayer McKinney, Christine McLeavey, Paul McMillan, Jake McNeil, David Medina, Aalok Mehta, Jacob Menick, Luke Metz, Andrey Mishchenko, Pamela Mishkin, Vinnie Monaco, Evan Morikawa, Daniel Mossing, Tong Mu, Mira Murati, Oleg Murk, David Mély, Ashvin Nair, Reiichiro Nakano, Rajeev Nayak, Arvind Neelakantan, Richard Ngo, Hyeonwoo Noh, Long Ouyang, Cullen O'Keefe, Jakub Pachocki, Alex Paino, Joe Palermo, Ashley Pantuliano, Giambattista Parascandolo, Joel Parish, Emy Parparita, Alex Passos, Mikhail Pavlov, Andrew Peng, Adam Perelman, Filipe de Avila Belbute Peres, Michael Petrov, Henrique Ponde de Oliveira Pinto, Michael, Pokorny, Michelle Pokrass, Vitchyr H. Pong, Tolly Powell, Alethea Power, Boris Power, Elizabeth Proehl, Raul Puri, Alec Radford, Jack Rae, Aditya Ramesh, Cameron Raymond, Francis Real, Kendra Rimbach, Carl Ross, Bob Rotsted, Henri Roussez, Nick Ryder, Mario Saltarelli, Ted Sanders, Shibani Santurkar, Girish Sastry, Heather Schmidt, David Schnurr, John Schulman, Daniel Selsam, Kyla Sheppard, Toki Sherbakov, Jessica Shieh, Sarah Shoker, Pranav Shyam, Szymon Sidor, Eric Sigler, Maddie Simens, Jordan Sitkin, Katarina Slama, Ian Sohl, Benjamin Sokolowsky, Yang Song, Natalie Staudacher, Felipe Petroski Such, Natalie Summers, Ilya Sutskever, Jie Tang, Nikolas Tezak, Madeleine B. Thompson, Phil Tillet, Amin Tootoonchian, Elizabeth Tseng, Preston Tuggle, Nick Turley, Jerry Tworek, Juan Felipe Cerón Uribe, Andrea Vallone, Arun Vijayvergiya, Chelsea Voss, Carroll Wainwright, Justin Jay Wang, Alvin Wang, Ben Wang, Jonathan Ward, Jason Wei, CJ Weinmann, Akila Welihinda, Peter Welinder, Jiayi Weng, Lilian Weng, Matt Wiethoff, Dave Willner, Clemens Winter, Samuel Wolrich, Hannah Wong, Lauren Workman, Sherwin Wu, Jeff Wu, Michael Wu, Kai Xiao, Tao Xu, Sarah Yoo, Kevin Yu, Qiming Yuan, Wojciech Zaremba, Rowan Zellers, Chong Zhang, Marvin Zhang, Shengjia Zhao, Tianhao Zheng, Juntang Zhuang, William Zhuk, and Barret Zoph. 2024. GPT-4 Technical Report. arXiv:2303.08774 [cs.CL] https://arxiv.org/abs/2303.08774

[35] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. 2025. Splitwise: Efficient Generative LLM Inference Using Phase Splitting. In *Proceedings of the 51st Annual International Symposium on Computer Architecture* (Buenos Aires, Argentina) *(ISCA '24)*. IEEE Press, 118–132. https://doi.org/10.1109/ISCA59077.2024.00019

[36] Ruoyu Qin, Zheming Li, Weiran He, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. 2025. Mooncake: A KVCache-centric Disaggregated Architecture for LLM Serving. arXiv:2407.00079 [cs.DC] https://arxiv.org/abs/2407.00079

[37] Qwen Team. 2025. Qwen3-32B. ModelScope. https://www.modelscope.cn/models/Qwen/Qwen3-32B Accessed: 2025-05-20.

[38] Chaoyi Ruan, Yinhe Chen, Dongqi Tian, Yandong Shi, Yongji Wu, Jialin Li, and Cheng Li. 2025. DynaServe: Unified and Elastic Execution for Dynamic Disaggregated LLM Serving. arXiv:2504.09285 [cs.DC] https://arxiv.org/abs/2504.09285

[39] SGLang. 2025. *SGLang: A fast serving framework for large language models and vision-language models.* https://github.com/sgl-project/sglang Open-source under Apache-2.0 license.

[40] Rana Shahout, Eran Malach, Chunwei Liu, Weifan Jiang, Minlan Yu, and Michael Mitzenmacher. 2025. Don't Stop Me Now: Embedding Based Scheduling for LLMs. arXiv:2410.01035 [cs.LG] https://arxiv.org/abs/2410.01035

[41] ShareGPT Teams. 2023. ShareGPT. https://sharegpt.com/.

[42] Xiaoxiang Shi, Colin Cai, Junjia Du, Zhanda Zhu, Xingda Wei, and Zhihao Jia. 2025. Nexus: Proactive Intra-GPU Disaggregation of Prefill and Decode in LLM Serving. arXiv:2507.06608 [cs.DC] https://arxiv.org/abs/2507.06608

[43] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. 2020. Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism. arXiv:1909.08053 [cs.CL] https://arxiv.org/abs/1909.08053

[44] Gursimran Singh, Xinglu Wang, Yifan Hu, Timothy Yu, Linzi Xing, Wei Jiang, Zhefeng Wang, Xiaolong Bai, Yi Li, Ying Xiong, Yong Zhang, and Zhenan Fan. 2025. Efficiently Serving Large Multimodal Models Using EPD Disaggregation. arXiv:2501.05460 [cs.DC] https://arxiv.org/abs/2501.05460

[45] Jovan Stojkovic, Chaojie Zhang, Íñigo Goiri, Josep Torrellas, and Esha Choukse. 2025. DynamoLLM: Designing LLM Inference Clusters for Performance and Energy Efficiency. In *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 1348–1362. https://doi.org/10.1109/hpca61900.2025.00102

[46] Foteini Strati, Sara Mcallister, Amar Phanishayee, Jakub Tarnawski, and Ana Klimovic. 2024. DéjàVu: KV-cache Streaming for Fast, Fault-tolerant Generative LLM Serving. arXiv:2403.01876 [cs.DC] https://arxiv.org/abs/2403.01876

[47] Biao Sun, Ziming Huang, Hanyu Zhao, Wencong Xiao, Xinyi Zhang, Yong Li, and Wei Lin. 2024. Llumnix: Dynamic Scheduling for Large Language Model Serving. arXiv:2406.03243 [cs.AR] https://arxiv.org/abs/2406.03243

[48] 5 Team, Aohan Zeng, Xin Lv, Qinkai Zheng, Zhenyu Hou, Bin Chen, Chengxing Xie, Cunxiang Wang, Da Yin, Hao Zeng, Jiajie Zhang, Kedong Wang, Lucen Zhong, Mingdao Liu, Rui Lu, Shulin Cao, Xiaohan Zhang, Xuancheng Huang, Yao Wei, Yean Cheng, Yifan An, Yilin Niu, Yuanhao Wen, Yushi Bai, Zhengxiao Du, Zihan Wang, Zilin Zhu, Bohan Zhang, Bosi Wen, Bowen Wu, Bowen Xu, Can Huang, Casey Zhao, Changpeng Cai, Chao Yu, Chen Li, Chendi Ge, Chenghua Huang, Chenhui Zhang, Chenxi Xu, Chenzheng Zhu, Chuang Li, Congfeng Yin, Daoyan Lin, Dayong Yang, Dazhi Jiang, Ding Ai, Erle Zhu, Fei Wang, Gengzheng Pan, Guo Wang, Hailong Sun, Haitao Li, Haiyang Li, Haiyi Hu, Hanyu Zhang, Hao Peng, Hao Tai, Haoke Zhang, Haoran Wang, Haoyu Yang, He Liu, He Zhao, Hongwei Liu, Hongxi Yan, Huan Liu, Huilong Chen, Ji Li, Jiajing Zhao, Jiamin Ren, Jian Jiao, Jiani Zhao, Jianyang Yan, Jiaqi Wang, Jiayi Gui, Jiayue Zhao, Jie Liu, Jijie Li, Jing Li, Jing Lu, Jingsen Wang, Jingwei Yuan, Jingxuan Li, Jingzhao Du, Jinhua Du, Jinxin Liu, Junkai Zhi, Junli Gao, Ke Wang, Lekang Yang, Liang Xu, Lin Fan, Lindong Wu, Lintao Ding, Lu Wang, Man Zhang, Minghao Li, Minghuan Xu, Mingming Zhao, Mingshu Zhai, Pengfan Du, Qian Dong, Shangde Lei, Shangqing Tu, Shangtong Yang, Shaoyou Lu, Shijie Li, Shuang Li, Shuang-Li, Shuxun Yang, Sibo Yi, Tianshu Yu, Wei Tian, Weihan Wang, Wenbo Yu, Weng Lam Tam, Wenjie Liang, Wentao Liu, Xiao Wang, Xiaohan Jia, Xiaotao Gu, Xiaoying Ling, Xin Wang, Xing Fan, Xingru Pan, Xinyuan Zhang, Xinze Zhang, Xiuqing Fu, Xunkai Zhang, Yabo Xu, Yandong Wu, Yida Lu, Yidong Wang, Yilin Zhou, Yiming Pan, Ying Zhang, Yingli Wang, Yingru Li, Yinpei Su, Yipeng Geng, Yitong Zhu, Yongkun Yang, Yuhang Li,

Yuhao Wu, Yujiang Li, Yunan Liu, Yunqing Wang, Yuntao Li, Yuxuan Zhang, Zezhen Liu, Zhen Yang, Zhengda Zhou, Zhongpei Qiao, Zhuoer Feng, Zhuorui Liu, Zichen Zhang, Zihan Wang, Zijun Yao, Zikang Wang, Ziqiang Liu, Ziwei Chai, Zixuan Li, Zuodong Zhao, Wenguang Chen, Jidong Zhai, Bin Xu, Minlie Huang, Hongning Wang, Juanzi Li, Yuxiao Dong, and Jie Tang. 2025. GLM-4.5: Agentic, Reasoning, and Coding (ARC) Foundation Models. arXiv:2508.06471 [cs.CL] https://arxiv.org/abs/2508.06471

[49] Kimi Team, Yifan Bai, Yiping Bao, Guanduo Chen, Jiahao Chen, Ningxin Chen, Ruijue Chen, Yanru Chen, Yuankun Chen, Yutian Chen, Zhuofu Chen, Jialei Cui, Hao Ding, Mengnan Dong, Angang Du, Chenzhuang Du, Dikang Du, Yulun Du, Yu Fan, Yichen Feng, Kelin Fu, Bofei Gao, Hongcheng Gao, Peizhong Gao, Tong Gao, Xinran Gu, Longyu Guan, Haiqing Guo, Jianhang Guo, Hao Hu, Xiaoru Hao, Tianhong He, Weiran He, Wenyang He, Chao Hong, Yangyang Hu, Zhenxing Hu, Weixiao Huang, Zhiqi Huang, Zihao Huang, Tao Jiang, Zhejun Jiang, Xinyi Jin, Yongsheng Kang, Guokun Lai, Cheng Li, Fang Li, Haoyang Li, Ming Li, Wentao Li, Yanhao Li, Yiwei Li, Zhaowei Li, Zheming Li, Hongzhan Lin, Xiaohan Lin, Zongyu Lin, Chengyin Liu, Chenyu Liu, Hongzhang Liu, Jingyuan Liu, Junqi Liu, Liang Liu, Shaowei Liu, T. Y. Liu, Tianwei Liu, Weizhou Liu, Yangyang Liu, Yibo Liu, Yiping Liu, Yue Liu, Zhengying Liu, Enzhe Lu, Lijun Lu, Shengling Ma, Xinyu Ma, Yingwei Ma, Shaoguang Mao, Jie Mei, Xin Men, Yibo Miao, Siyuan Pan, Yebo Peng, Ruoyu Qin, Bowen Qu, Zeyu Shang, Lidong Shi, Shengyuan Shi, Feifan Song, Jianlin Su, Zhengyuan Su, Xinjie Sun, Flood Sung, Heyi Tang, Jiawen Tao, Qifeng Teng, Chensi Wang, Dinglu Wang, Feng Wang, Haiming Wang, Jianzhou Wang, Jiaxing Wang, Jinhong Wang, Shengjie Wang, Shuyi Wang, Yao Wang, Yejie Wang, Yiqin Wang, Yuxin Wang, Yuzhi Wang, Zhaoji Wang, Zhengtao Wang, Zhexu Wang, Chu Wei, Qianqian Wei, Wenhao Wu, Xingzhe Wu, Yuxin Wu, Chenjun Xiao, Xiaotong Xie, Weimin Xiong, Boyu Xu, Jing Xu, Jinjing Xu, L. H. Xu, Lin Xu, Suting Xu, Weixin Xu, Xinran Xu, Yangchuan Xu, Ziyao Xu, Junjie Yan, Yuzi Yan, Xiaofei Yang, Ying Yang, Zhen Yang, Zhilin Yang, Zonghan Yang, Haotian Yao, Xingcheng Yao, Wenjie Ye, Zhuorui Ye, Bohong Yin, Longhui Yu, Enming Yuan, Hongbang Yuan, Mengjie Yuan, Haobing Zhan, Dehao Zhang, Hao Zhang, Wanlu Zhang, Xiaobin Zhang, Yangkun Zhang, Yizhi Zhang, Yongting Zhang, Yu Zhang, Yutao Zhang, Yutong Zhang, Zheng Zhang, Haotian Zhao, Yikai Zhao, Huabin Zheng, Shaojie Zheng, Jianren Zhou, Xinyu Zhou, Zaida Zhou, Zhen Zhu, Weiyu Zhuang, and Xinxing Zu. 2025. Kimi K2: Open Agentic Intelligence. arXiv:2507.20534 [cs.LG] https://arxiv.org/abs/2507.20534

[50] Qwen Team. 2024. Qwen2.5: A Party of Foundation Models. https://qwenlm.github.io/blog/qwen2.5/

[51] Tencent Hy Team. 2026. Hy3-preview. https://github.com/Tencent-Hunyuan/Hy3-preview. GitHub repository, accessed: 2026-06-01.

[52] Lean Wang, Huazuo Gao, Chenggang Zhao, Xu Sun, and Damai Dai. 2024. Auxiliary-Loss-Free Load Balancing Strategy for Mixture-of-Experts. arXiv:2408.15664 [cs.LG] https://arxiv.org/abs/2408.15664

[53] Zhibin Wang, Zetao Hong, Xue Li, Zibo Wang, Shipeng Li, Qingkai Meng, Qing Wang, Chengying Huan, Rong Gu, Sheng Zhong, and Chen Tian. 2025. Adaptive Rescheduling in Prefill-Decode Disaggregated LLM Inference. arXiv:2510.13668 [cs.DC] https://arxiv.org/abs/2510.13668

[54] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. 2023. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. arXiv:2201.11903 [cs.CL] https://arxiv.org/abs/2201.11903

[55] Bingyang Wu, Yinmin Zhong, Zili Zhang, Shengyu Liu, Fangyue Liu, Yuanhang Sun, Gang Huang, Xuanzhe Liu, and Xin Jin. 2024. Fast Distributed Inference Serving for Large Language Models. arXiv:2305.05920 [cs.LG] https://arxiv.org/abs/2305.05920

[56] Yu Wu, Yicheng Feng, Yibo Jin, Zhuoran Yang, Zhenyu Ji, Wenhao Li, Ruihao Gong, Yibo Zhu, Yuhao Zhu, et al. 2025. Arrow: Adaptive Scheduling Mechanisms for Disaggregated LLM Inference Architecture. arXiv:2505.11916 [cs.DC] https://arxiv.org/abs/2505.11916

[57] Yuxing Xiang, Xue Li, Kun Qian, Yufan Yang, Diwen Zhu, Wenyuan Yu, Ennan Zhai, Xuanzhe Liu, Xin Jin, and Jingren Zhou. 2025. Aegaeon: Effective GPU Pooling for Concurrent LLM Serving on the Market. In *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles* (Lotte Hotel World, Seoul, Republic of Korea) *(SOSP '25)*. Association for Computing Machinery, New York, NY, USA, 1030–1045. https://doi.org/10.1145/3731569.3764815

[58] Tairan Xu, Zhiyuan Fang, Xianwei Zhang, Jiawei Mi, Yingchun Wang, and Junyi Zhu. 2025. Prefill-Decode Aggregation or Disaggregation? Unifying Both for Goodput-Optimized LLM Serving. arXiv:2508.01989 [cs.DC] https://arxiv.org/abs/2508.01989

[59] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. 2025. Qwen3 Technical Report. arXiv:2505.09388 [cs.CL] https://arxiv.org/abs/2505.09388

[60] Zed Industries. 2024. Zeta: A Dataset for Predictive Code Editing. Hugging Face Datasets. https://huggingface.co/datasets/zed-industries/zeta Accessed: 2025-12-02.

[61] Dingyan Zhang, Haotian Wang, Yang Liu, Xingda Wei, Yizhou Shan, Rong Chen, and Haibo Chen. 2025. BlitzScale: Fast and Live Large Model Autoscaling with O(1) Host Caching. In *19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25)*. USENIX Association, Boston, MA, 275–293. https://www.usenix.org/conference/osdi25/presentation/zhang-dingyan

[62] Yue Zhang, Yuansheng Chen, Xuan Mo, Alex Xi, Jialun Li, and WeiGang Wu. 2025. A Predictive and Synergistic Two-Layer Scheduling Framework for LLM Serving. arXiv:2509.23384 [cs.DC] https://arxiv.org/abs/2509.23384

[63] Chenqi Zhao, Wenfei Wu, Linhai Song, Yuchen Xu, and Yitao Yuan. 2026. Fine-grained MoE Load Balancing with Linear Programming. arXiv:2511.16947 [cs.DC] https://arxiv.org/abs/2511.16947

[64] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, Yifan Du, Chen Yang, Yushuo Chen, Zhipeng Chen, Jinhao Jiang, Ruiyang Ren, Yifan Li, Xinyu Tang, Zikang Liu, Peiyu Liu, Jian-Yun Nie, and Ji-Rong Wen. 2025. A Survey of Large Language Models. arXiv:2303.18223 [cs.CL] https://arxiv.org/abs/2303.18223

[65] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving. arXiv:2401.09670 [cs.DC]

APPENDIX

A Why Tail Latency Matters?

In production LLM serving, relying solely on central tendency metrics (such as Mean or P50) often creates a false sense of security regarding system performance. Figure 1 illustrates a typical latency distribution observed under standard scheduling policies. While the median (P50) TPOT comfortably satisfies the stringent SLO of 100ms (indicated by the red dashed line), the distribution exhibits a severe long-tail phenomenon. As shown, the tail latencies degrade exponentially: the P99 latency exceeds 1500ms, and the P99.9 latency spikes to over 2500ms, more than **25× higher** than the SLO threshold. For interactive applications like chatbots, these tail latencies manifest as noticeable "stuttering" during text generation, severely disrupting the user experience. Consequently, optimizing for tail latency (P99/P99.9) rather than P50 is critical for ensuring consistent quality of service, motivating the design of Sibyl to specifically mitigate these worst-case scenarios.

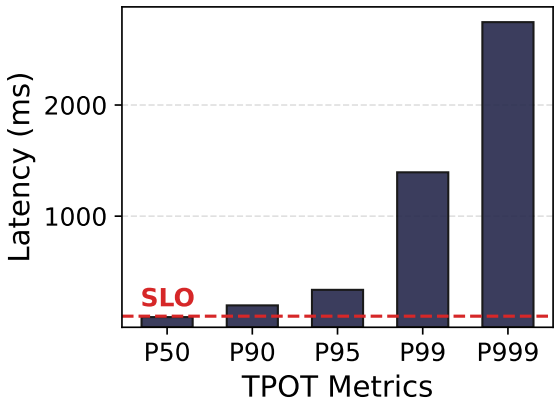


Figure 1. The long-tail latency problem. *Although the median (P50) TPOT meets the 100ms SLO (red dashed line), severe congestion causes P99 and P99.9 latencies to violate the constraint by orders of magnitude, highlighting the inadequacy of P50 as a sole metric.*

B Temporal Consistency

Our formulation is designed to be *boundary-consistent* across state transitions to ensure scheduling stability. First, at the *Active-Shadow boundary* (Type 1 ↔ Type 2), a task transitioning from queue to execution shifts from using the system-average speed ($\bar{v}_{sys}$) to its specific rate (v_k). Mathematically, since v_k is initially unknown, $\bar{v}_{sys}$ serves as its unbiased estimator at the boundary. While the realization of v_k during execution may introduce a value update, the *expected load* remains statistically consistent at the instant of transition.

Second, at the *Shadow-Decay boundary* (Type 2 ↔ Type 3), the model naturally converges: as the projected start time aligns with the handoff ($\tau_k \to \tau$), the generation component vanishes, and both Type 2 and Type 3 models resolve identically to the intrinsic input cost ($\lim_{\tau_k \to \tau} \mathcal{L} = l_k^{in}$). This design eliminates singularity points in the objective function, allowing the scheduler to smoothly track tasks as they drift through different lifecycle stages.

Table 1. *Generation hyperparameters used to enable reasoning mode.*

Parameter	Value
Temperature	0.6
Top-P	0.95
Top-K	20
Min-P	0
Greedy Decoding	Disabled

C Detailed Experimental Setup

C.1 Model Configurations

To enable the reasoning capabilities of Qwen3-8B and avoid deterministic degeneration, we disable greedy decoding and employ stochastic sampling. We adopt the official hyperparameters recommended by Qwen Team [37], as detailed in Table 1.

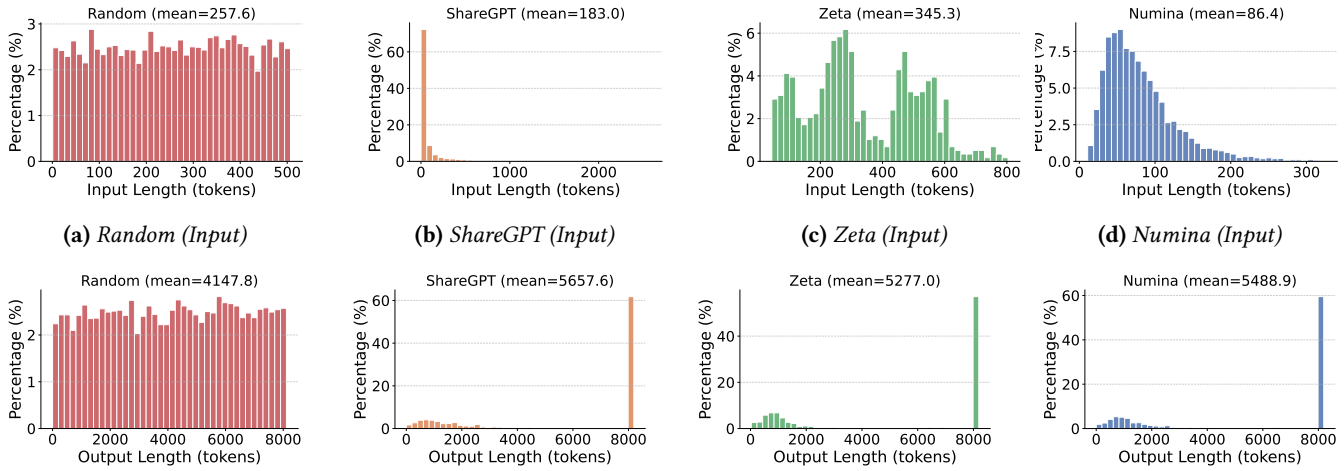
C.2 Dataset Details

We evaluate our system using four datasets that cover synthetic micro-benchmarks, real-world conversations, code generation tasks, and mathematical reasoning. The input and output length distributions are visualized in Figure 2.

Random. This synthetic dataset is designed to micro-benchmark system performance under reasoning-intensive scenarios (*e.g.,* Chain-of-Thought), where short questions trigger long responses. We uniformly sample input prompt lengths from $[1, 512]$ and output lengths from $[1, 8192]$. This configuration stresses the decode phase and allows us to test the scheduler's behavior under uniform workload variance.

ShareGPT. Representing real-world human-chatbot interactions, this dataset features relatively short prompts (mean length 183) but exhibits high variance in response lengths. Under our reasoning-enabled configuration, the output distribution becomes extremely heavy-tailed; as shown in Figure 2(e), about 62% of requests saturate the maximum token limit (8192), creating sustained pressure on decode instances.

Zeta. Sourced from predictive code editing tasks, this dataset introduces longer context prefixes compared to S hareGPT, with input lengths extending up to 1.7k tokens (mean 345). Similar to ShareGPT, the output distribution is dominated by long-running tasks that frequently hit the generation cap (57% reach 8192 tokens), effectively testing the system's ability to handle heavier prefill loads alongside concurrent long-context decoding.



(a) *Random (Input)* **(b)** *ShareGPT (Input)* **(c)** *Zeta (Input)* **(d)** *Numina (Input)*

(e) *Random (Output)* **(f)** *ShareGPT (Output)* **(g)** *Zeta (Output)* **(h)** *Numina (Output)*

Figure 2. Token length distributions across datasets. *The top row shows the input prompt length distributions, while the bottom row shows the output generation length distributions for Random, ShareGPT, Zeta, and Numina.*

Numina. Drawn from competition-level mathematical problems, this dataset pairs concise problem statements (mean input length 86) with lengthy step-by-step reasoning traces. The output distribution is the most decode-heavy among all datasets, with an average response length of roughly 5.5k tokens and nearly 60% of requests exhausting the 8192-token budget, making it a stringent test for sustained decode throughput under minimal prefill cost.

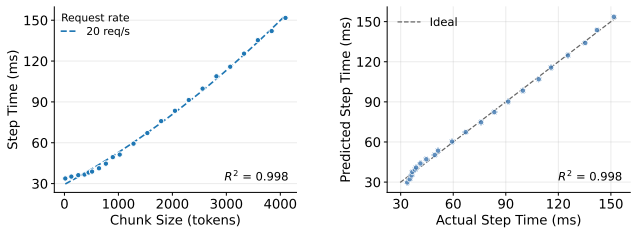
```
# Execution Command for Prefill Instance
vllm serve <MODEL_PATH> \
    --host <HOST_IP> \
    --port <HTTP_PORT> \
    --served-model-name qwen3-8b \
    --tensor-parallel-size 1 \
    --seed 1024 \
    --dtype float16 \
    --max-model-len 32768 \
    --max-num-batched-tokens 32768 \
    --max-num-seqs 1 \
    --trust-remote-code \
    --reasoning-parser qwen3 \
    --gpu-memory-utilization 0.9 \
    --compilation-config '{"cudagraph_mode": "PIECEWISE"}' \
    --kv-transfer-config '{
        "kv_connector": "P2pNcclConnector",
        "kv_role": "kv_producer",
        "kv_buffer_size": "1e1",
        "kv_port": "<KV_PORT>",
        "kv_connector_extra_config": {
            "proxy_ip": "<PROXY_IP>",
            "proxy_port": "<PROXY_PORT>",
            "http_port": "<HTTP_PORT>"
        }
    }'
```

```
# Execution Command for Decode Instance
vllm serve <MODEL_PATH> \
    --host <HOST_IP> \
    --port <HTTP_PORT> \
    --served-model-name qwen3-8b \
    --tensor-parallel-size 1 \
    --seed 1024 \
    --dtype float16 \
    --max-model-len 32768 \
    --max-num-batched-tokens 32768 \
    --max-num-seqs 256 \
    --trust-remote-code \
    --reasoning-parser qwen3 \
    --gpu-memory-utilization 0.8 \
    --kv-transfer-config '{
        "kv_connector": "P2pNcclConnector",
        "kv_role": "kv_consumer",
        "kv_buffer_size": "8e9",
        "kv_port": "<KV_PORT>",
        "kv_connector_extra_config": {
            "proxy_ip": "<PROXY_IP>",
            "proxy_port": "<PROXY_PORT>",
            "http_port": "<HTTP_PORT>"
        }
    }'
```

```
# Benchmarking Command
vllm bench serve \
    --backend vllm \
    --model qwen3-8b \
    --tokenizer <MODEL_PATH> \
    --dataset-name "hf" \
    --dataset-path <DATASET_PATH> \
    --hf-name <DATASET_NAME> \
    --host <PROXY_IP> \
    --port <PROXY_PORT> \
    --burstiness <BURSTINESS> \
    --percentile-metrics "ttft,tpot,itl,e2el" \
    --metric-percentiles "50,90,95,99,99.9" \
    --seed $(date +%s) \
    --trust-remote-code \
    --request-rate <RPS> \
    --num-prompts <NUM_PROMPTS>
```

C.3 vLLM Runtime Configuration

We deploy the disaggregated inference system using vLLM. Specifically, we configure the system with the `P2pNcclConnector`, which natively supports the overlapping of prefill computation with KV cache transmission. The following listings provide the representative command-line templates used in our experiments. Environment-specific variables (*e.g.*,



(a) *Step time vs. chunk size* **(b)** *Actual vs. predicted*

Figure 3. Offline fitting of the prefill per-step latency. *(a) Per-step latency scales smoothly with chunk size and is fit by a low-degree polynomial ($R^2 = 0.998$). (b) Predicted step time closely matches the measured values.*

IPs, ports) and dynamic workload parameters have been replaced with placeholders for generality.

C.4 Prefill Step-Time Fitting

As described in §5, Sibyl estimates a request's prefill completion time from its number of chunked-prefill steps and a per-step latency model profiled offline on the target hardware. Figure 3(a) shows that, with `batch_size=1` and chunked prefill, the per-step latency grows smoothly with the chunk size and is well captured by a low-degree polynomial ($R^2 = 0.998$). Figure 3(b) confirms that the fitted model closely tracks the measured step time across the operating range. This lightweight model lets the proxy translate a prompt length into an accurate handoff-time estimate at negligible cost.

D Additional Experiment Results

The primary evaluation in the main text utilizes NVIDIA H800 GPUs. To validate the broad applicability and robustness of Sibyl across diverse hardware architectures, this section supplements those results with experiments on two additional platforms (Table 2): the NVIDIA H20, a decode-optimized accelerator with massive HBM capacity (141 GB) and high bandwidth (4.0 TB/s) but relatively lower compute power (148 TFLOPS, BF16), making it exceptionally suitable for the memory-bound decode phase of LLM inference; and the NVIDIA A100, a flagship predecessor with significantly higher compute capability (312 TFLOPS, BF16) but more constrained memory resources (40 GB HBM, 1.89 TB/s).

Table 2. *Hardware specification comparison among NVIDIA H20, A100, and H800 GPUs.*

Specification	NVIDIA H20	NVIDIA A100	NVIDIA H800
FP16/BF16 Compute	148 TFLOPS	312 TFLOPS	1,979 TFLOPS
HBM Capacity	141 GB	40 GB	80 GB
HBM Bandwidth	4.0 TB/s	1.89 TB/s	3.35 TB/s

D.1 Experimental Setup

NVIDIA H20. We deploy Sibyl on a server node with eight NVIDIA H20 GPUs and evaluate Qwen3-32B [59], which fits within the H20's 141 GB HBM without tensor parallelism [43]. To accommodate the varying prefill–decode ratios of different workloads, we adopt a 2P4D topology (2 prefill and 4 decode instances) for the Random dataset and a 4P4D topology for the ShareGPT and Zeta datasets, with each GPU hosting a single model replica.

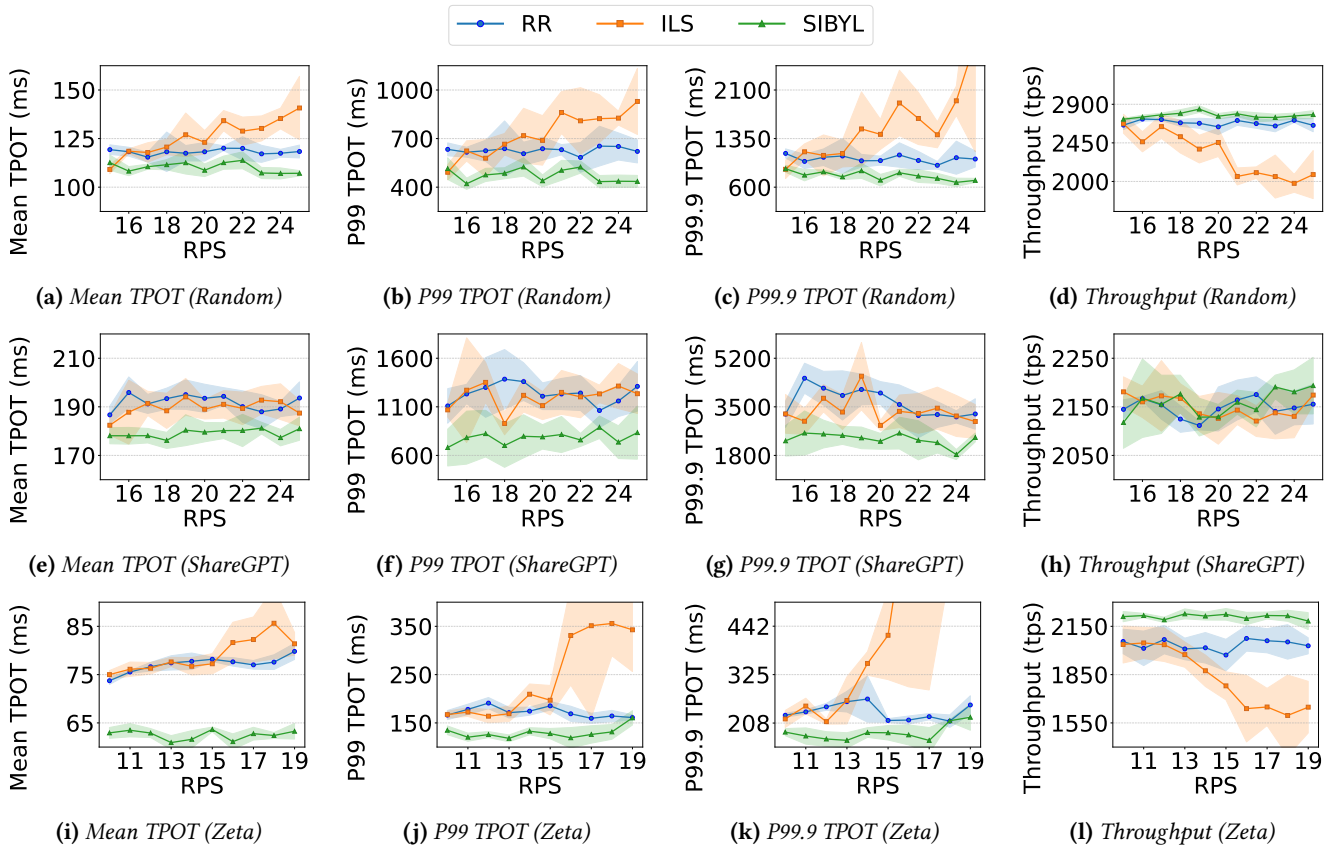
NVIDIA A100. We further conduct experiments on a distributed cluster of two physical servers equipped with NVIDIA A100 GPUs, deployed in a 2P4D topology where the prefill workload runs on a 2×A100 server and the decode workload on a separate 4×A100 server, with each GPU hosting a single model replica without tensor parallelism. Here we evaluate two widely adopted open-weights models, Qwen2.5-7B-Instruct [50] and Llama3-8B [2], on a synthetic random dataset whose input prompt lengths are uniformly distributed in $[1, 1024]$ and output generation lengths in $[1, 4096]$.

Across both platforms, we compare Sibyl against the RR and ILS baselines, mirroring the deployable-scheduler comparison of the main text.

D.2 Overall Performance

D.2.1 NVIDIA H20. Figure 4 summarizes the end-to-end performance of Sibyl, ILS, and RR with Qwen3-32B across three representative workloads on the H20 platform. First, on the bursty Random workload, Sibyl increases output-token throughput by 1.21× (vs. ILS) and 1.03× (vs. RR), while reducing Mean TPOT by 12.0% (vs. ILS) and 6.7% (vs. RR). Notably, it effectively suppresses tail latency, lowering P99 and P99.9 TPOT by 32.7% and 43.4% (vs. ILS), as well as by 24.5% and 25.2% (vs. RR). Second, on the real-world ShareGPT dataset, while throughput remains comparable across methods due to system saturation (∼1.0×), Sibyl stabilizes generation performance by mitigating hotspots: it reduces Mean TPOT by ∼6% and cuts tail latency drastically, lowering P99 TPOT by 34.3% (vs. ILS) and 36.3% (vs. RR), and P99.9 TPOT by 28.6% (vs. ILS) and 34.8% (vs. RR). Finally, on the Zeta dataset, Sibyl achieves its most pronounced gains, boosting throughput by 1.22× (vs. ILS) and 1.09× (vs. RR) and reducing Mean TPOT by 20.7% (vs. ILS) and 18.9% (vs. RR). More importantly, Sibyl achieves strict SLO compliance with massive reductions in tail latency, where P99 and P99.9 TPOT drop by 41.9% and 50.6% (vs. ILS), as well as 24.2% and 20.8% (vs. RR). These results collectively show that Sibyl's predictive scheduling not only maximizes cluster capacity in bursty scenarios but also ensures prompt responsiveness in production-grade workloads.

D.2.2 NVIDIA A100. Figure 5 illustrates the end-to-end performance comparison on the random dataset. We observe

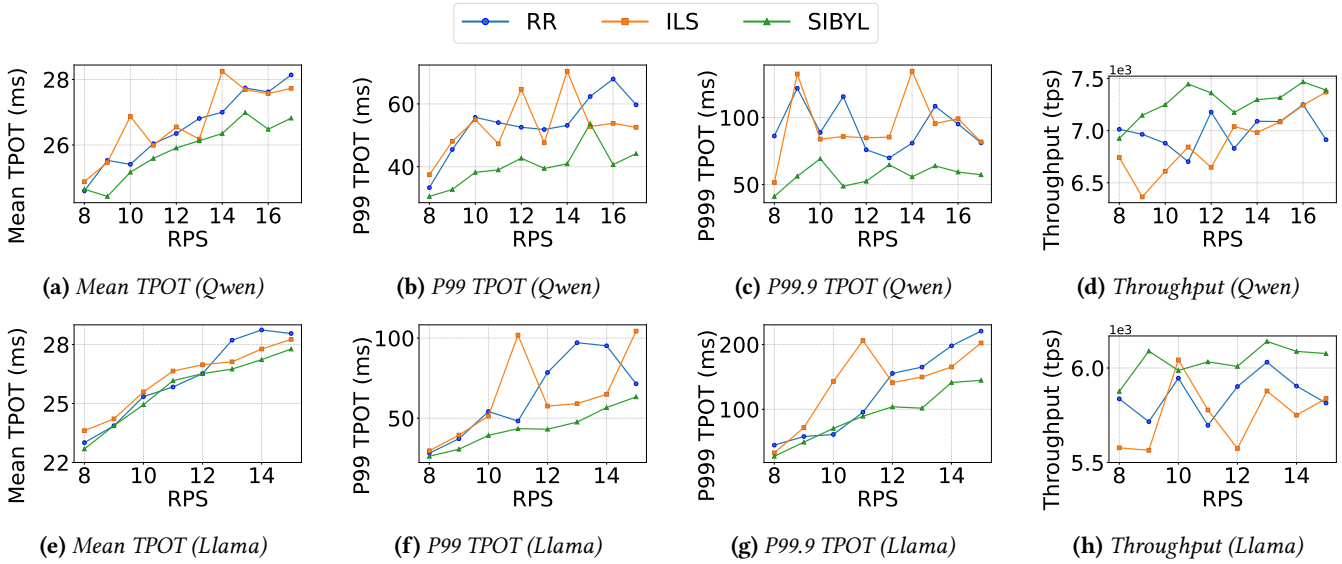


(a) *Mean TPOT (Random)* **(b)** *P99 TPOT (Random)* **(c)** *P99.9 TPOT (Random)* **(d)** *Throughput (Random)*

(e) *Mean TPOT (ShareGPT)* **(f)** *P99 TPOT (ShareGPT)* **(g)** *P99.9 TPOT (ShareGPT)* **(h)** *Throughput (ShareGPT)*

(i) *Mean TPOT (Zeta)* **(j)** *P99 TPOT (Zeta)* **(k)** *P99.9 TPOT (Zeta)* **(l)** *Throughput (Zeta)*

Figure 4. *End-to-end performance comparison of* Sibyl*, ILS, and RR with Qwen3-32B [59] on NVIDIA H20 across the Random, ShareGPT, and Zeta workloads.*



(a) *Mean TPOT (Qwen)* **(b)** *P99 TPOT (Qwen)* **(c)** *P99.9 TPOT (Qwen)* **(d)** *Throughput (Qwen)*

(e) *Mean TPOT (Llama)* **(f)** *P99 TPOT (Llama)* **(g)** *P99.9 TPOT (Llama)* **(h)** *Throughput (Llama)*

Figure 5. *End-to-end performance comparison of Qwen2.5-7B-Instruct [50] and Llama3-8B [2] on random dataset.*

that Sibyl consistently outperforms both RR and ILS baselines across different models and varying request rates (RPS), particularly in suppressing tail latency and improving system throughput.

Qwen2.5-7B Results. As shown in the top row of Figure 5, SIBYL demonstrates robust stability under high contention. Compared to ILS, SIBYL reduces the average P99 and P99.9 TPOT by **23.0%** and **36.2%**, respectively, with peak reductions reaching up to **58.6%** in the most congested scenarios. Similarly, against RR, SIBYL achieves average reductions of **24.2%** (P99) and **36.4%** (P99.9). In terms of throughput, SIBYL improves the average throughput by **5.7%** over ILS and **4.2%** over RR, effectively handling higher loads without violating latency constraints.

Llama3-8B Results. The bottom row of Figure 5 confirms the generalizability of our approach. SIBYL achieves significant tail latency improvements, reducing the average P99 TPOT by **26.4%** vs. ILS and **26.2%** vs. RR. For the strict P99.9 metric, SIBYL lowers the latency by an average of **32.0%** compared to ILS. Throughput gains remain consistent, with SIBYL outperforming ILS by an average of **5.1%** and RR by **3.1%**.

While the improvement in Mean TPOT is modest (approximately 2–3% across workloads), the substantial reduction in tail latency (P99/P99.9) indicates that SIBYL effectively mitigates the severe congestion instances caused by suboptimal scheduling. Crucially, this suppression of tail latency directly translates to significantly fewer SLO violations, thereby guaranteeing a higher quality of service.

E Simulation Methodology

To evaluate SIBYL at scales exceeding typical physical testbeds (e.g., 64–256 decode instances) and to conduct extensive stress testing under varied arrival rates, we implemented a high-fidelity, trace-driven discrete-event simulator. This section details the simulation environment, the hardware modeling assumptions, and the implementation logic corresponding to Algorithms 2 and 3.

E.1 Hardware Modeling

Unlike basic simulators that assume constant processing speeds, our model explicitly accounts for memory bandwidth contention. Since LLM decoding is bandwidth-bound, we model the GPU as a shared resource where the total throughput is divided among active requests. Consequently, as the number of concurrent tasks increases, the execution speed of each individual request decreases, accurately reflecting the performance slowdown in real hardware.

Dynamic Throughput. The total system throughput (TPS_{total}) of a decode instance is not static; it exhibits nonlinear variations governed by the request concurrency (N) and the total token volume (T). Instead of relying on simple univariate polynomial fitting, we accurately model these hardware dynamics using a *Random Forest Regressor* trained on profiling data from the NVIDIA H20 GPU (visualized in Figure 6). The model predicts throughput as the ensemble

Algorithm 2: Trace-Driven Discrete-Event Simulation Framework

Input : Request Trace $\mathcal{T}$, Cluster Config C, Scheduler $\mathcal{S}$
Output : Latency Metrics, Throughput

Initialize N Decode instances, global `RequestTable` (for SIBYL), and set $t_{now} \leftarrow 0$;
for *each request* r *in* $\mathcal{T}$ *sorted by arrival time* **do**
  $t_{now} \leftarrow r.arrival\_time$;
  for *each instance* $j \in \{1 \dots N\}$ **do**
    AdvanceInstance(j, t_{now}) ; // See Algorithm 3
    Update instance speed v_j based on dynamic load;
  if $\mathcal{S}$ *is SIBYL* **then**
    Sync `RequestTable` with physical state;
    $\delta_{pre} \leftarrow$ EstimatePrefill$(r.input_len)$;
    $\tau \leftarrow t_{now} + \delta_{pre}$;
    $j^* \leftarrow \arg\min_j$ ProjectedLoad(j, τ);
  else if $\mathcal{S}$ *is ILS* **then**
    $j^* \leftarrow \arg\min_j$ CurrentLoad(j, t_{now});
  else
    $j^* \leftarrow (j_{last} + 1) \pmod N$;
  $t_{start\_decode} \leftarrow t_{now} + \delta_{pre}$;
  Push event `START_DECODE(r)` to instance j^* queue at t_{start_decode};
Flush all instances until completion;

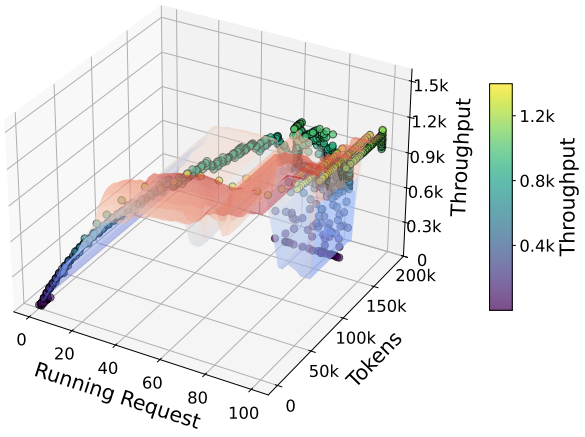


Figure 6. *TPS profiling on NVIDIA H20 GPU.*

average of 100 decision trees:

$$TPS_{total}(N, T) = \frac{1}{100} \sum_{i=1}^{100} \text{Tree}_i(N, T) \tag{18}$$

Evaluation on a hold-out test set demonstrates high fidelity with an R^2 of 0.9024. Feature importance analysis reveals that

while concurrency is the dominant factor (85.51%), the total token load exerts a significant influence (14.49%), validating the necessity of a bi-variate model to capture memory bandwidth saturation effects. In our simulation (Algorithm 3, Line 4), we query this inference model to dynamically recalibrate instance speed based on the real-time state (N, T).

Bandwidth Contention. When multiple requests decode concurrently, they share the GPU's memory bandwidth. The simulator models this by splitting the total throughput equally among all active tasks (Algorithm 3, Line 5). Consequently, the effective speed v_{speed} of a single request decreases as the batch size increases, accurately simulating the slowdown effect caused by multiple parallel requests.

E.2 Simulation Logic and Workflow

The simulation proceeds as a strict discrete-event system driven by the request arrival trace. The interaction between the high-level scheduling logic and the low-level physics engine is detailed below:

1. Global Event Loop (Algorithm 2). The framework processes requests sequentially based on their arrival times. Upon each arrival at time t_{now}, the system first ensures state synchronization (Lines 6-9) by invoking the AdvanceInstance subroutine for every instance; this advances the physical simulation to the current timestamp, processing any interim task completions or new decoding starts. With the system state synchronized, the scheduler then executes the decision-making logic (Lines 11-17) to select a target instance j^*. For Sibyl, this involves syncing the shadow `RequestTable` and projecting loads at the future handoff time τ. Crucially, to explicitly model the temporal gap (Lines 19-20), the request is not executed immediately; instead, a `START_DECODE` event is registered in the target instance's queue, scheduled to trigger strictly after the prefill duration at $t_{now} + \delta_{pre}$.

2. Instance State Evolution (Algorithm 3). This subroutine acts as the physics engine. It advances the state of a specific instance continuously. Instead of fixed time steps, it calculates Δt_{step} (Line 9) as the minimum time until the next discrete event: either a running task finishes its generation (Δt_{finish}) or a pending prefill task enters the decode phase (Δt_{start}). This event-driven progression ensures that resource contention is recalculated exactly when the system load changes, guaranteeing high temporal precision.

Algorithm 3: Instance State Evolution (Processor Sharing Model)

1 **Function** *AdvanceInstance*$(instance, t_{target})$
  // Simulates continuous execution up to t_{target}
2 **while** $instance.time < t_{target}$ **do**
3 $Q_{run} \leftarrow instance.decoding\_tasks$;
4 $L_{tokens} \leftarrow \sum_{k \in Q_{run}} (len_{prompt,k} + len_{generated,k})$;
5 $TPS_{total} \leftarrow$ GetFittedThroughput(Q_{run}, L_{tokens}) ; // Based on profiling curve (Figure 6)
6 $v_{speed} \leftarrow TPS_{total}/|Q_{run}|$; // Processor sharing: split equally
  // Determine time to next state change
7 $\Delta t_{finish} \leftarrow \min_{k \in Q_{run}} (remaining\_tokens_k / v_{speed})$;
8 $\Delta t_{start} \leftarrow instance.next\_start\_event - instance.time$;
9 $\Delta t_{step} \leftarrow \min(\Delta t_{finish}, \Delta t_{start}, t_{target} - instance.time)$;
  // Advance physical state
10 **for** *each task* k *in* Q_{run} **do**
11 $remaining\_tokens_k \leftarrow remaining\_tokens_k - (\Delta t_{step} \times v_{speed})$;
12 $instance.time \leftarrow instance.time + \Delta t_{step}$;
  // Handle discrete events
13 **if** $\Delta t_{step} == \Delta t_{finish}$ **then**
14 Identify task k^* with $remaining \leq 0$;
15 Remove k^* from Q_{run}, record t_{end} and TPOT;
16 **else if** $\Delta t_{step} == \Delta t_{start}$ **then**
17 Pop task r_{new} from event queue;
18 Add r_{new} to Q_{run} with initial remaining load;

F Definitions of Sensitivity Variants

In this section, we provide precise definitions for the variants evaluated in the sensitivity analysis (Section 6.4). To isolate the contribution of each modeling component in Sibyl, we modify the objective function or the load estimation logic as follows:

Prob@1 (No Probabilistic Weighting): This variant disables the survival analysis module. Instead of computing the conditional survival probability based on historical distributions, we assume a deterministic worst-case scenario where every active task persists until the completion of the incoming request's prefill phase. Mathematically, we set the probability term $\mathcal{P} = 1$ in Eq. (15) and Eq. (16). This baseline evaluates the necessity of risk-adjusted load estimation versus conservative over-provisioning.

w/o Type 1 (No Active Task Projection): We exclude the projected load from currently running active tasks by setting $\mathcal{L}_{\mathrm{I}}(k, \tau) = 0$. This variant forces the scheduler to ignore the residual workload on decode instances, relying solely on the

estimated pressure from pending (shadow) tasks. This tests whether knowing the state of currently executing requests is critical for decision-making.

w/o Type 2 (No Pre-Handoff Shadow): We ignore the "invisible" contention from requests that are currently in the prefill phase but are projected to start decoding *before* the target request arrives ($\tau_k \leq \tau$). By setting $\mathcal{L}_{\text{II}}(k, \tau) = 0$, the scheduler fails to account for the queue that will form during the prefill interval. This baseline highlights the impact of the temporal gap described in §2.2.

w/o Type 3 (No Post-Handoff Shadow): We exclude the interference decay model for tasks projected to start *after* the handoff moment ($\tau_k > \tau$). By setting $\mathcal{L}_{\text{III}}(k, \tau) = 0$, the scheduler optimizes only for the expected load at moment τ, ignoring imminent future arrivals that might cause congestion shortly after.

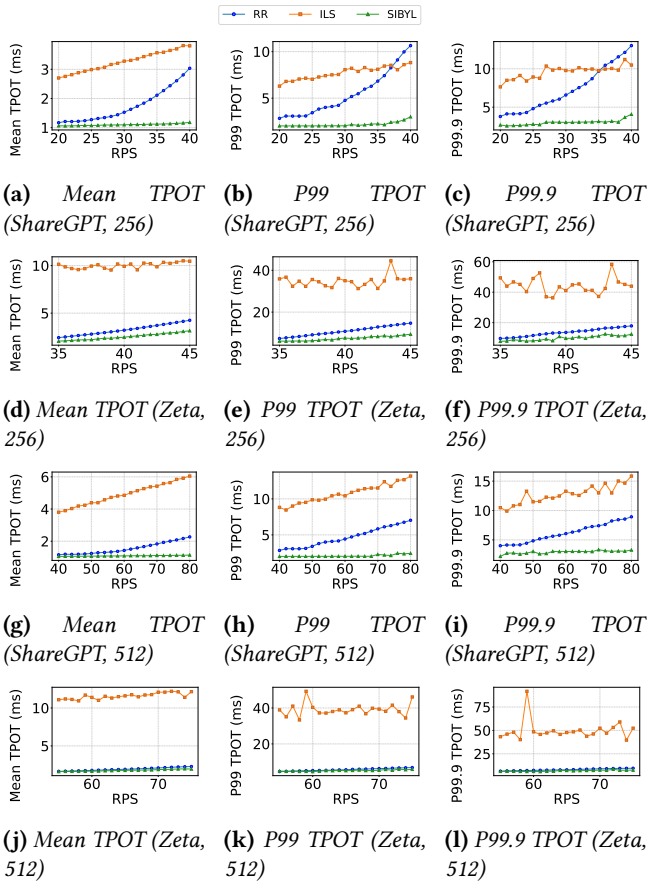
Req Level (Request-Level Balancing): This variant abandons the token-based load metric entirely. Instead of modeling physical memory pressure (token occupancy) and duration, it treats all requests as identical units. The scheduler assigns the new request to the instance with the fewest total tasks, regardless of their generation lengths or memory footprint. Notably, unlike the ILS baseline, this variant explicitly accounts for the count of pending requests currently in the prefill phase. This serves as a baseline to validate the necessity of fine-grained, token-aware modeling in heterogeneous LLM workloads.

G Extended Simulation Results

To verify the scalability of Sibyl in massive large-scale environments, we extend our simulation to clusters with 256 and 512 decode instances. Figure 7 presents the latency characteristics under sweeping arrival rates.

Latency Stability at Scale. Sibyl demonstrates robust scalability, maintaining a flat latency profile even as the system approaches saturation. In contrast, the benchmark methods suffer from varying degrees of performance degradation. On the heavy-tailed ShareGPT workload, RR blindly routes long requests to busy instances, causing unbalanced load. Sibyl effectively suppresses this tail latency, outperforming RR by 54.33% (256 instances) and 51.66% (512 instances) in P99 TPOT, while surpassing ILS by 71.73% and 80.21% respectively. On the Zeta dataset, the gap widens further as ILS struggles with long prefill durations that create large temporal gaps. This "invisible load" issue renders ILS's instantaneous view obsolete, causing severe congestion. Consequently, Sibyl achieves drastic reductions in P99 TPOT against ILS: 78.65% on 256 instances and 86.66% on 512 instances, confirming that predictive lookahead is critical for scaling coding workloads.

Mechanistic Explanation via Assignment Accuracy. Table 3 elucidates the root cause of these performance gaps.



(a) *Mean TPOT (ShareGPT, 256)* **(b)** *P99 TPOT (ShareGPT, 256)* **(c)** *P99.9 TPOT (ShareGPT, 256)*

(d) *Mean TPOT (Zeta, 256)* **(e)** *P99 TPOT (Zeta, 256)* **(f)** *P99.9 TPOT (Zeta, 256)*

(g) *Mean TPOT (ShareGPT, 512)* **(h)** *P99 TPOT (ShareGPT, 512)* **(i)** *P99.9 TPOT (ShareGPT, 512)*

(j) *Mean TPOT (Zeta, 512)* **(k)** *P99 TPOT (Zeta, 512)* **(l)** *P99.9 TPOT (Zeta, 512)*

Figure 7. *Simulation performance on scaled clusters with 256 and 512 decode instances.*

Table 3. *Optimal assignment ratio comparison across different cluster scales.*

Dataset	# Inst.	RR	ILS	Sibyl
ShareGPT	256	56.6%	30.6%	**90.6%**
	512	61.7%	19.9%	**91.6%**
Zeta	256	29.1%	10.1%	**40.4%**
	512	50.0%	0.08%	**57.5%**

Sibyl achieves superior precision, maintaining > 90% accuracy on ShareGPT and significantly outperforming baselines on Zeta. Notably, ILS frequently underperforms the load-agnostic RR (*e.g.*, 10.1% vs. 29.1% on Zeta-256). This counter-intuitive result confirms that acting on *stale* information (ILS) creates severe "herding," where multiple requests are routed to the same seemingly idle instance, whereas Sibyl's predictive lookahead effectively resolves this contention.